IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

SPECIAL SECTION ON IEEE EDUCATION SOCIETY SECTION

Received 1 April 2025, accepted 29 April 2025, date of publication 1 May 2025, date of current version 14 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3566230

TUTORIAL

Local, Global, and Saga Transactions for SOA Services and Microservices (*Services)

ANTONIO NAVARRO
Departamento de Ingeniería del Software e Inteligencia Artificial, Facultad de Informática, Universidad Complutense de Madrid, 28040 Madrid, Spain
e-mail: anavarro@fdi.ucm.es

ABSTRACT Transactions are a key issue in enterprise applications. Considering SOA and microservices as some of the most prevalent solutions for enterprise applications, transaction management differs significantly between these two solutions. This paper defines MOF metamodels for characterizing transaction management in SOA services and microservices. To obtain these metamodels, the paper also defines additional MOF metamodels: one for the main components of the multitier architecture, one for the hexagonal architecture, one that relates both architectures, one for SOA services, and one for microservices. The paper also includes the design of a simple transaction manager that illustrates the details of how to implement global transactions for SOA services. This transaction manager, or other simpler ones, can also be used with microservices. In addition, several types of transactions are analyzed and implemented in SOA services and microservices to compare the differences between both approaches. The paper is intended as a guide for those lecturers with subjects focused on enterprise application architectures. It can also be used by developers looking for a detailed comparison between SOA services and microservices to decide which is best for their needs.

INDEX TERMS SOAP, WSDL, REST, XA transaction, distributed transaction, multitier architecture, hexagonal architecture, MSA.

I. INTRODUCTION

Remote services are one of the main strategies for implementing enterprise applications (EAs) today. Although there are various types of remote services, this paper focuses on three main types: web services, REST services and microservices. Web services are based on WSDL and SOAP standards, whereas REST services are based on HTTP protocol. Microservices adhere to the principles of *microservice architecture* (MSA) and are usually deployed on ephemeral containers. This is an arguable division of remote services, especially considering that microservices are usually exposed via a REST API. The next section discusses this classification, but for now, this paper groups SOAP and non-MSA REST services together as *SOA (Service-Oriented Architecture) services* and leave MSA services (or microservices) in their own category. Table 1 summarizes the main differences between SOA and microservices in enterprise applications using RDBMSs for data storage.

The associate editor coordinating the review of this manuscript and approving it for publication was James Harland.

Categories aside, whenever remote services are described or classified, the emphasis is not usually on service transactions, even though remote services usually perform transactional operations against transactional resources. This means that, to some extent, these resources must: (i) agree that the operation has been successfully performed or not; (ii) maintain some type of data consistency; (iii) have isolation among concurrent operations; (iv) have durability in their operations; and (v) ideally, a group of operations could be atomically performed against the resources. Atomic, consistent, isolated and durable transactions are called *ACID transactions* [1].

ACID transactions mark the difference between SOA services and microservices: unless the transaction is long-running (i.e. it is going to involve a significant amount of time that will not allow the whole transaction to be closed in the same moment), the SOA participants try to make an atomic, consistent, isolated and durable transaction, whereas microservices, regardless of the transaction duration, try to execute their transaction in an independent manner, informing the other participants (*choreography*) or a director (*orchestration*) of the result of this transaction, in order to

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

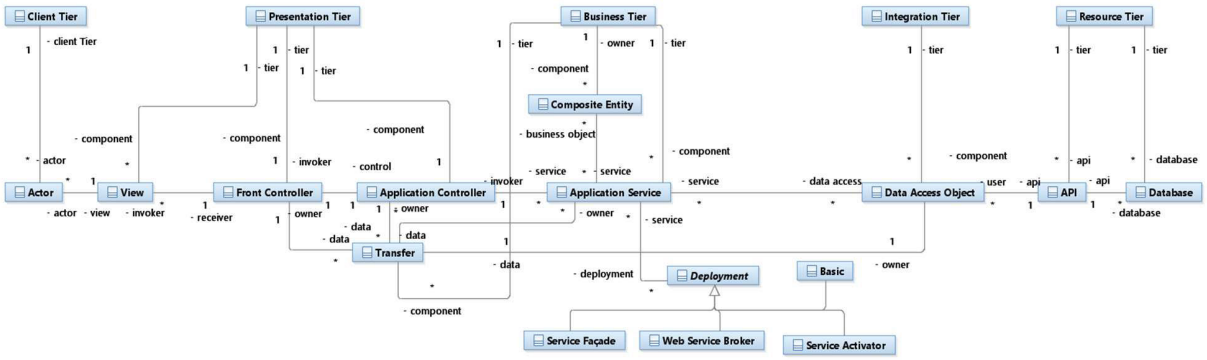


FIGURE 1. MOF metamodel for the main components of the multitier architecture.

TABLE 1. Main diffrences between SOA and microservices in prototypical enterprise applications.

Characteristic	*SOA*	*Microservices*
Typical deployment	Application server in virtual machine	Services in Docker container or serverless deployment
Granularity for business logic at design	*Application service* [1] (e.g. department application service)	Application service (e.g. department application service)
Granularity for business logic at deployment	Application as a sum of application services in an application server (e.g. banking application)	Independent application services deployed in independent Docker containers or serverless deployment (e.g., department service, employee service, account service, etc.)
Granularity for database at design level	Domain database (e.g. bank database)	Entity bound to the application service (e.g. department, employee, account, etc.)
Granularity for database at deployment level	Domain database in single database schema in single (or cluster) RDBMS (e.g. bank database schema)	Multiple database schemas (one per entity) deployed in single (or cluster) RDBMS (e.g. department schema, employee schema, account schema, etc.)

maintain some sort of atomicity and consistency at present or in the future. This type of transaction management, where the system may eventually become inconsistent is called *eventual consistency*, also known as BASE (Basically Available, Soft State and Eventually consistent). *Basically available* because the system is available most of the time; *soft state* because data can have temporary states that can evolve without external inputs to achieve consistency; and *eventually consistent* because data inconsistencies will be fixed at some point.

The main reason for this difference is the way that SOA services and microservices store data: SOA services usually access a single database with all the data necessary to perform an operation, while microservices only access the data related to the service itself.

Of course, SOA services can perform transactions with several participants scattered across several databases all over the world, but these services try to create the illusion of handling the scattered transaction as a unique transaction. They perform a *global transaction* [1].

When dealing with microservices, each service usually acts only against one entity, and even operations against two related entities (e.g., a client and its bank account) are split into two independent transactions against two databases.

As a result, SOA services require complex transaction managers capable of coordinating the transaction in each participant as a unified transaction and simple services that perform operations against their transactional resources, regardless of whether their transaction is single or part of a larger transaction. On the contrary, microservices need simple transaction managers that perform their transaction in an individual manner, but they need a complex infrastructure of asynchronous messaging to inform other participants in the transaction how their own transaction has finished.

For example, let us suppose a simple use case of client registration in a bank. In this example, every active user must have an active account. The use case gathers the client data and creates the user and a new account with a balance of 0$, if there is not a client with that e-mail in the bank. The user may wish to open several accounts, or an account may have several clients, but for simplicity, the use case considers that only one client with one account stored in a *Relational Database Management System* (RDBMS).

An SOA service can start a transaction against the RDBMs, check the client, insert the client and the account into two different tables of the same database schema, and close the transaction while keeping atomicity (e.g. both entities are created or none), consistency (e.g. the client referenced by the account exists and is active), isolation (no changes performed by other transaction in clients or accounts can be seen in the current transaction), and durability (if the entities are inserted, they will stay in the database). If, for some reason, clients and accounts are deployed in two different RDBMSs, both participants try to perform an atomic, consistent, isolated and durable transaction using a global transaction, and in case of fatal failure, an inconsistency is produced, and some compensation transaction is executed.

Because entities and accounts are different entities, the microservice architecture considers two different tables in two different database schemas (and probably two different RDBMSs). The service responsible for user registration takes care of recording the user in its table, but it needs the presence of an account service to record the account data in its table. Both services perform individual transactions for record-

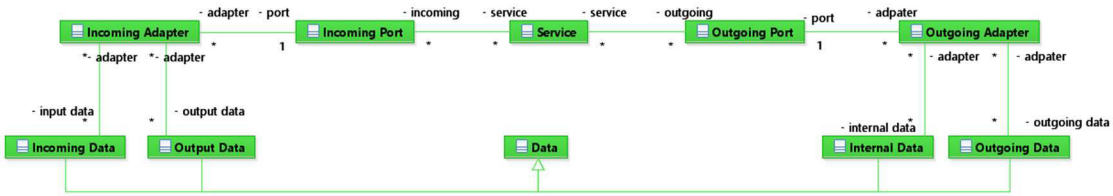


FIGURE 2. MOF metamodel for the hexagonal architecture.

ing the data. If something goes wrong during the account recording and nothing is done, an active client is going to be recorded without an active account, violating the bank's policy. Thus, services must use *sagas* [1] and asynchronous messaging to *inform the interested services* of what they are doing so that they can run *compensating transactions* [1] to resolve any inconsistencies that their operations may have produced.

This leads to changes in both the transactional infrastructure and the business logic of microservices with regard to SOA services, and is by far, excluding the splitting of data among databases, or precisely because of it, the most striking difference between the two types of services.

However, as in Table 1, descriptions of SOA services and microservices tend to focus on questions about deployment, asynchronous messaging, or single database per service, without emphasizing one of the most noticeable differences between SOA services and microservices: transaction management.

This paper focuses on service transaction management by reviewing the literature on this topic (Section II), providing a MOF metamodel that gathers the main concepts regarding service transactions (Section III), providing a simple transaction manager for SOA services (Section IV), describing the difference in the transaction management between SOA services and microservices (Section V), discussing the findings (Section VI), and presenting conclusions (Section VII).

The main goal is to serve as a guide for lecturers responsible for teaching subjects focused on enterprise application architectures, complementing the information about transaction management included in *ACM/IEEE Computer Science Curricula 2023* [2] and the information provided in my previous paper [1]. It can also be used by developers looking for a detailed comparison between SOA services and microservices to decide which is best for their needs. Thus, the paper provides new teaching resources and better real-world applicability for those interested in service transactions. To this end, the paper has the following contributions: (i) it defines MOF metamodels that structure and illustrate the main concepts behind multitier architecture, hexagonal architecture, the relationships between both architectures, SOA services, and microservices; (ii) extends the list of transaction managers defined in previous work [1] providing a new transaction manager able to deal with global transactions performed in the context of web services; and (iii) provides five detailed examples that illustrate the differences between SOA and microservices when dealing with transaction management, as they use a completely different approach, ACID transaction management for SOA services and eventual consistency for microservices.

II. RELATED WORK

A. SOA SERVICES TRANSACTIONS

Excellent literature regarding transaction management exists, such as [3], [4] and [5].

My previous paper reviews the main concepts behind transaction management, including transaction managers for local and global transactions based on the *two phase-commit* (2PC) protocol [1]. It also includes a brief discussion on transaction management for SOA services and microservices. Additionally, the paper introduces the concept of *service transaction*: a transaction started and finished from a business service, particularly an application service [1]. This concept includes local transactions invoked from a web application and is therefore more general than the SOA and microservice transactions analyzed in this paper.

Web service transactions based on application servers are usually implemented using WS-* standards, such as *WS-Atomic* [6] and *WS-Business Activity* [7] for dealing with atomic (ACID local and global transactions) and long-running transactions respectively. Both are based on the generic standard for activity management in the web services *WS-Coordination* [8]. *WS-Transactions* gathers these standards [9]. Service transactions built on them usually perform a declarative transaction management which hides the transaction-related code [1]. WS-Coordination identifies three main elements bound to the *coordinator*, the service responsible for coordinating services: an *activation service*, a *registration service* and a coordination-type-specific set of *coordination protocols*. The idea behind WS-Coordination is very accurate: in an enterprise transaction involving several web services, it is necessary: (i) to have a coordinator that takes care of the global transaction; (ii) to have a unique transaction identifier that is used to coordinate the global transaction in each service (activation service); (iii) to have some mechanisms to inform the coordinator who are the participants invoked by the application service that started the activation of the transaction (registration service); (iv) to have a commonly agreed protocol to identity and perform

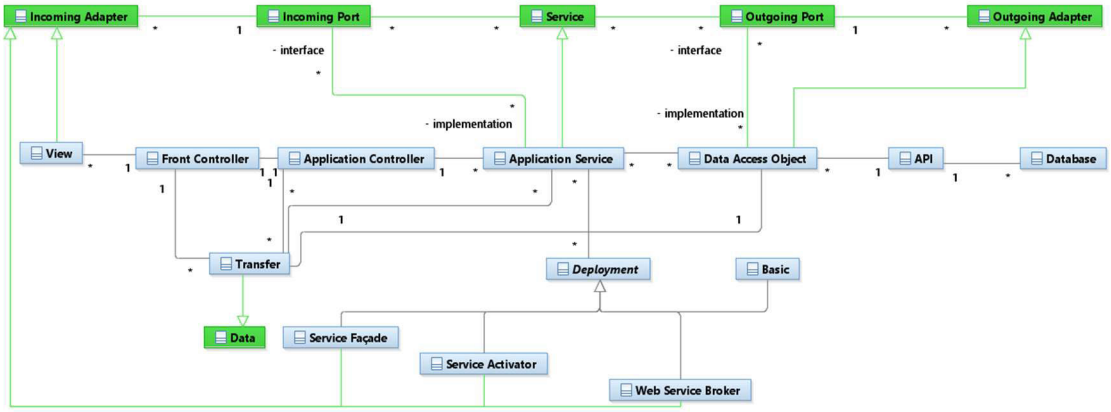


FIGURE 3. Mapping between the MOF metamodels for the multitier and hexagonal architecture.

the type of transaction being performed (2PC, long-running, etc.); and (v) to have a transaction *context*, originating from the activation service, that flows among services carrying information about the transaction identifier and the registration service or the coordinator of the invoking service. WS-Coordination is a very short standard that provides information only about the SOAP messages for supporting it. WS-Atomic and WS-Business Activity are very succinct also. They basically describe the state machine that dictates the states a transaction can have according to these protocols. Long and complex standards such as *HyTime* [10], which are hard to understand and implement, are doomed to fail. However, the concepts behind WS-Coordination, WS-Atomic and WS-Business Activity are very complex for the length of these standards. They are minimal documents that define a global agreement among the implementors of application servers. Therefore, their contribution to the understanding of transaction management for web services is almost nonexistent. The documentation regarding these standards in the application servers is also not very detailed.

On the contrary, the *JBoss Transactions Development Guide* can be the best example for understanding these WS standards in an application server [11]. Reference [12] provides a good description of protocols for long-running business transactions.

Focusing on academic references, [13] provides a formal model of transaction for web services, but it is very theoretical and far from being usable from the point of view of developers. Reference [14] has a similar approach. Reference [15] provides a data modelling and contracting framework for transactional service composition, but also from a theoretical perspective. Reference [16] only provides a brief description of WS-Business Activity. Reference [17] proposes a modelling methodology based on UML which considers four concerns: structure, transactions, workflow and security, but the transaction concern is not developed in depth. Reference [18] presents OpenWS-Transaction, an open-source middleware based on WS-Transactions. Reference [19] defines a UML model for business transactions and focuses on transactional requirements between business participants, but detailed examples of transaction processing are not included. Reference [20] provides a design of compensations using forward recovery within web service transactions but again, from a high-level perspective. Reference [21] proposes a mechanism to ensure the consistent executions of isolation-relaxing web service transactions, but also from a high-level perspective. Reference [22] analyzes web service transactions from several points of view but does not provide detailed examples of these transactions. Reference [23] provides a high-level framework for ensuring consistency of web services transactions. Reference [24] provides a high-level coordination framework for fast atomic transactions using web services.

There is some controversy about the inclusion of global transactions in REST web services. However, [25] and [26] are good references on this issue. Reference [25] focuses on the feasibility of global transactions on REST services and [26] defines how to perform atomic transactions using REST principles. Reference [27] focuses on asynchronous transactions based on JMS but is also very abstract.

Thus, references on web services transactions are basically descriptions of the implementation of WS-Transaction in application servers or theoretical papers on transactional models for web services. In the REST domain, most papers aim to show that global transactions fit in the REST philosophy.

This paper focuses specifically on the transaction management in SOA services, providing metamodels for classifying both these services and their transactions, and comparing them with microservices transactions. It also provides a simple transaction manager capable of managing global transactions in SOA services. Finally, detailed examples of transaction management in SOA services for different types of transactions (local, global and sagas) are provided and compared with their counterparts in microservice implementation.

B. MICROSERVICES TRANSACTIONS

A common approach for handling consistency across microservices is *eventual consistency* [1]. This model does not enforce global ACID transactions across microservices. Instead, it suggests using some mechanisms to ensure that the system will be eventually consistent at some point in the future [28], [29]. Thus, according to the eventual consistency model an employee could be, at any given moment, registered in the system with reference to a deregistered department. However, at some point this problem will be detected and either the employee will be deregistered or the department will be activated. To undo these operations *compensating transactions* must be performed [28]. Compensating transactions are functions that, as their name suggests, have the task of returning the application to a consistent state by undoing the transactions that led the system into an inconsistent state.

Other options for dealing with data consistency between microservices include avoiding transactions across microservices (if possible) [29] or the two-phase commit protocol, which is not recommended [30], [31], [32].

The *saga pattern* was originally intended for long-running transactions that cannot be resolved in a small amount of time [33]. This pattern takes care of actual state of the transaction and uses compensating transactions to enforce consistency in case of eventual inconsistency.

Although most transactions performed by microservices are not long-running, the lone behavior of microservices regarding transaction management has led to the widespread use of sagas across microservices to deal with the eventual consistency, whether transactions are long-running or not [30], [31], [32], [34], [35].

As mentioned before, transaction managers for sagas are extremely simple, as they only need to deal with local transactions. However, the responsibility for maintaining consistency in the application data falls on the application side, which has to deal with a complex network of events exchanged across a network of asynchronous channels. References [30] and [36] are very good references for understanding sagas for microservices. However, these books use classes that belong to proprietary APIs, which hinders the understanding of the saga implementation. That is, the books focus on teaching the use of proprietary frameworks for implementing sagas instead of just implementing sagas using *POJO*s (Plain Old Java Objects). This problem also happens in the proprietary frameworks responsible for the implementation of sagas, such as those identified at [37]: *Axon Saga* [38], *Eclipse MicroProfiles LRA* [39], *Seata* [40], *Camunda* [41] and *Apache Camel* [42]. Reference [43] is a good survey about these frameworks. Reference [44] is an excellent reference for analyzing distributed transaction patterns for microservices beyond sagas.

Focusing on references with academic origin, González-Aparicio [45] describe a transaction platform for microservices-based big data systems, but it is a proprietary platform. Reference [46] provides an abstract model for reaching consensus in decentralized coordination for distributed microservices from a high-level perspective. Reference [47] describes a new control protocol for microservices transactions, but from a high-level perspective. Reference [48] focuses on a theoretical model for handling rollbacks in microservice architecture. Reference [49] describes a proprietary framework for the implementation of transactional microservices for financial applications.

This paper focuses specifically on transaction management in microservices, providing metamodels for classifying both these microservices and their transactions, and comparing them with SOA transactions. In addition, detailed examples of transaction management in microservices for different types of transactions (local, global and sagas) are provided and compared with their counterparts in SOA implementation. The description of the saga implementation omits the use of specific frameworks for the sake of clarity.

C. SUMMARY

In summary, the problems of present literature are related to complexity, abstraction and lack of comparison.

The concepts behind *service transactions are extremely complex. For example, the list of 53 definitions included in [1] provides an extensive enumeration of concepts related to global transactions. However, these concepts, as well as those provided in the analyzed references, lack any structure and the only relationship among them is some kind of implicit hyperlinked referencing. On the contrary, the metamodels defined in this paper provide a graphical representation of these concepts as well as their relationships.

The examples provided in these references are either too abstract or too specific for concrete frameworks. Therefore, it is difficult to achieve the ability to provide code from the abstract references or to induce general abstract concepts from the concrete frameworks. This paper is structured at a double level: metamodels provide abstract descriptions that are put into practice with concrete examples. In addition, no reference provides enough insights to understand the specific behavior of a transaction manager for SOA services, in contrast to the one defined in this paper.

Finally, references describe SOA services and microservices focusing only on one of them, particularly in terms of transaction management. Therefore, unlike this work, it is difficult to compare the advantages and disadvantages of both approaches when dealing with transactions.

III. MOF METAMODELS FOR MULTITIER ARCHITECTURE, HEXAGONAL ARHICTECTURE, *SERVICES AND *SERVICE TRANSACTION

Terminology is a very important issue when presenting key concepts. My previous paper provides an important set of definitions for dealing with service transactions [1]. In this paper, **services* are defined as an specific type that encompasses both SOA and microservices. In addition, the paper enhances my previous paper [1] by providing a more detailed

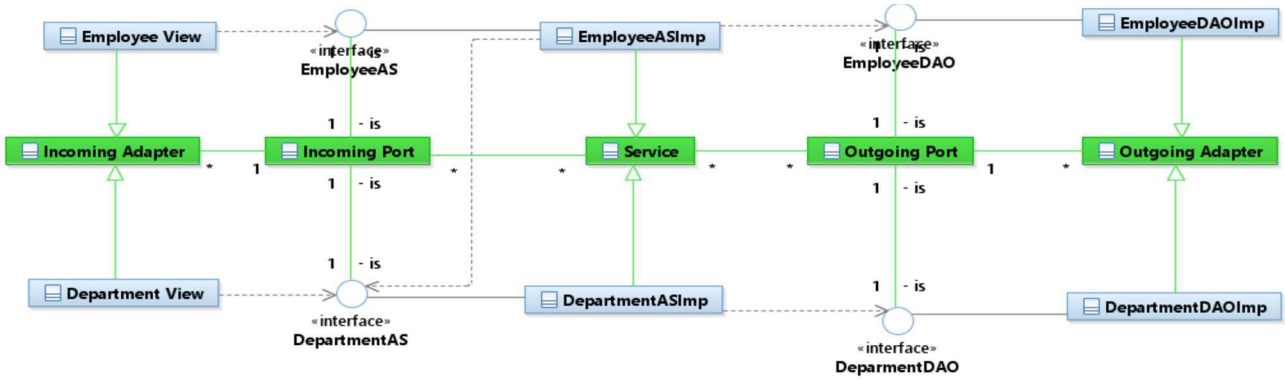


FIGURE 4. An example of multitier model (blue) and its relationship with the hexagonal metamodel (green). The model is idealized, as the interfaces of the application services should be exposed through web service brokers.

explanation of *service transaction management in terms of several metamodels, as well as a discussion about multitier [50], [51] and hexagonal architectures [30], [52].

The presence of these metamodels provides a graphical representation of the main domains under study, depicting the relationship between the components of each domain. Therefore, they are tools that improve the understanding of the domains as well as the implementation of transactions in SOA and microservices. For example, the metamodel for microservices enumerates the different components that are required for dealing with transactions in this domain in terms of concepts defined in multitier and hexagonal architectures. In this way, teachers, students and developers have a graphical enumeration of the classes necessary to implement transactions in microservices, as well as their relationships, as the examples provided in Section V exemplify.

A. MULTITIER ARCHITECTURE

The *multitier architecture* [50], [51] divides the application into five tiers: *user*, *presentation*, *business*, *integration* and *resource*. Because the user and resource tiers are external to the application, the architecture provides patterns for organizing the presentation, business and integration tiers. The presentation tier handles the incoming/outgoing information that flows between the user and the application, the business tier implements the business logic, and the integration tier manages the outgoing/incoming information that flows between the business tier and the resource tier (e.g., the database).

The multitier architecture catalogue has 25 patterns [50], and its description is outside the scope of this paper. Figure 1 depicts the MOF metamodel for the multitier architecture with the most relevant patterns for this paper.

As for the patterns depicted in Figure 1:

- *Front controller*: takes the HTTP-dependent information and translates it into HTTP-independent information. It is more of an *adapter* [53] than a controller.
- *Application controller*: a classic controller that uses *commands* [53] to make it reusable among applications.
- *Application service*: the core component responsible for implementing the business logic. Here is where the business logic of a use case is implemented, such as ensuring that a department exists and is active when a new employee (assigned to this department) is created.
- *Data Access Object* (*DAO*): a class that, irrespective of the business logic, reads/writes data from/to the database (resource tier, in general).
- *Transfer object*: a simple object-oriented representation of data. It has no operations beyond setters and getters and has no pointer to other objects that represent domain relationships. Thus, a transfer for an employee stores the identifier of its department, instead of a pointer to a transfer department. Transfers are key elements for sending information across tiers with a loose coupling among them. Therefore, no datatypes belonging to a specific tier (e.g., a text field belonging to the user interface) can be part of a transfer.
- *Service façade*: an RPC *proxy* [53] that gives access to an application service.
- *Web service broker*: an SOA service proxy that gives access to an application service.
- *Service activator*: a message-oriented proxy that gives access to an application service.

B. HEXAGONAL ARCHITECTURE

The *hexagonal architecture* [52] was *proposed* by Alistair Cockburn in 2005 [54] in order to "allow an application to equally be driven by users, programs, automated test or batch scripts, and to be developed and tested in isolation from its eventual run-time devices and databases" [52].

This architecture reduces the application to the business tier which communicates with the outer world via communication *ports*. Ports require *adapters* that translate protocol-dependent incoming information into application information, and outgoing information into protocol-dependent information. This results in a *left-right asymmetry* [52]: *primary/driving* ports and adapters take information from the outside and direct it to the application,

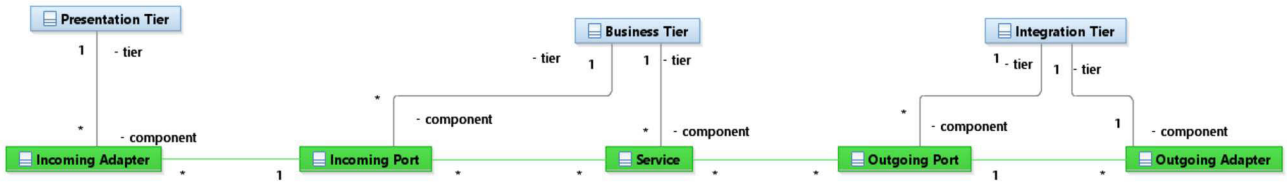


FIGURE 5. Alternative simple mapping between the MOF metamodels for the multitier and hexagonal architecture.

and *secondary/driven* ports take information from the application and direct it to the exterior.

Richardson calls *inbound* to the primary/driving ports and *outbound* to the secondary/driven ports [30]. The terms *incoming* and *outgoing* ports are used in this paper.

It is important to remark that the word *hexagonal* means nothing in the architecture: "the hexagon is not a hexagon because the number six is important, but rather to allow the people doing the drawing to have room to insert ports and adapters as they need, not being constrained by a one-dimensional layered drawing" [52].

A GUI can be an example of an incoming adapter; an application service interface can be an example of an incoming port; a data management interface can be an example of an outgoing port; and a database accessor can be an example of an outgoing adapter.

Figure 2 depicts the MOF metamodel for the hexagonal architecture. This paper uses *incoming data* to refer to the data that the application receives from a client, *output data* to refer to the data that the application responds to that client, *internal data* to refer to the data the application intends to send outside (not to the client), and *outgoing data* to refer to the data that the application sends outside (not to the client).

C. MAPPING BETWEEN MULTITIER AND HEXAGONAL ARCHITECTURES

Multitier architecture is the basis for SOA services [1] whilst hexagonal architecture is the basis for microservices [30]. Since both SOA services and microservices implement enterprise services, a reasonable question arises: what is the relationship between both architectures? Indeed, is there a mapping between both architectures? The mapping between the metamodels that characterize both architectures is depicted in Figure 3.

According to Figure 3, views are incoming adapters, application services are services, and the interfaces they implement are incoming ports, DAOs are outgoing adapters and the interfaces they implement are outgoing ports. Service façade, web service broker and service activators are incoming adapters. Transfers are data.

This mapping seems to be very convenient, but the example in Figure 4 raises some questions. This example is not very orthodox from the metamodeling point of view because it presents together elements belonging to the M2 tier (metamodel) and the M1 tier (model), but it is very descriptive. Elements belonging to M2 are depicted in green, and elements belonging to M1 are depicted in blue. This example presents two views, two application services and two DAOs responsible for managing employees and departments, and relates them to elements of the hexagonal architecture according to the mapping defined in Figure 3. However, a problem arises due to interservice communication. For example, let us suppose that the employee application service needs to communicate with the department application service to check if a department exists before creating a new employee, and if the department does not exist, it creates it. In this case, is the `DepartmentAS` interface an incoming or an outgoing port? It is clear that `DepartmentAS` is the basis mechanism for the application to communicate with the exterior regarding department management; therefore, it is an incoming port. But `EmployeeASImp` is not a *primary actor*: "an actor that drives the application (takes it out of quiescent state to perform one of its advertised functions)" [52]. And `EmployeeASImp` is invoking `DepartmentAS/DepartmentASImp`, which could create a new department in the database (using a DAO). Indeed, `DepartmentAS` seems to be a secondary actor: "one that the application drives, either to get answers from or to merely notify" [52]. Therefore, it can be considered an outgoing port, and thus, `DepartmentAS` is both an incoming and an outgoing port, and `DeparmentASImp` is both an element belonging to the core business logic application and an external outgoing adapter which sends information to the database. What happens? The problem here is the conception of *outside*(or primary) and *inside* (or secondary) that Cockburn conceives: "the distinction between primary and secondary lies in who triggers or is in charge of the conversation" [52]. The department AS can be both a component that responds to an actor or a component that responds to another component. Thus, the distinction between primary/driving/inbound/incoming and secondary/driven/outbound/outgoing gets blurred in an enterprise application and port adapters get something confusing unless you consider them only as a general-purpose object-oriented directive in order to obtain low coupling: use interfaces (ports) for isolate classes (applications components or adapters) that are in other packages.

Richardson tries to overcome this problem by considering incoming ports to the REST API that the services expose to the exterior to be invoked and solves the interservice communication with asynchronous messages. Publishers of

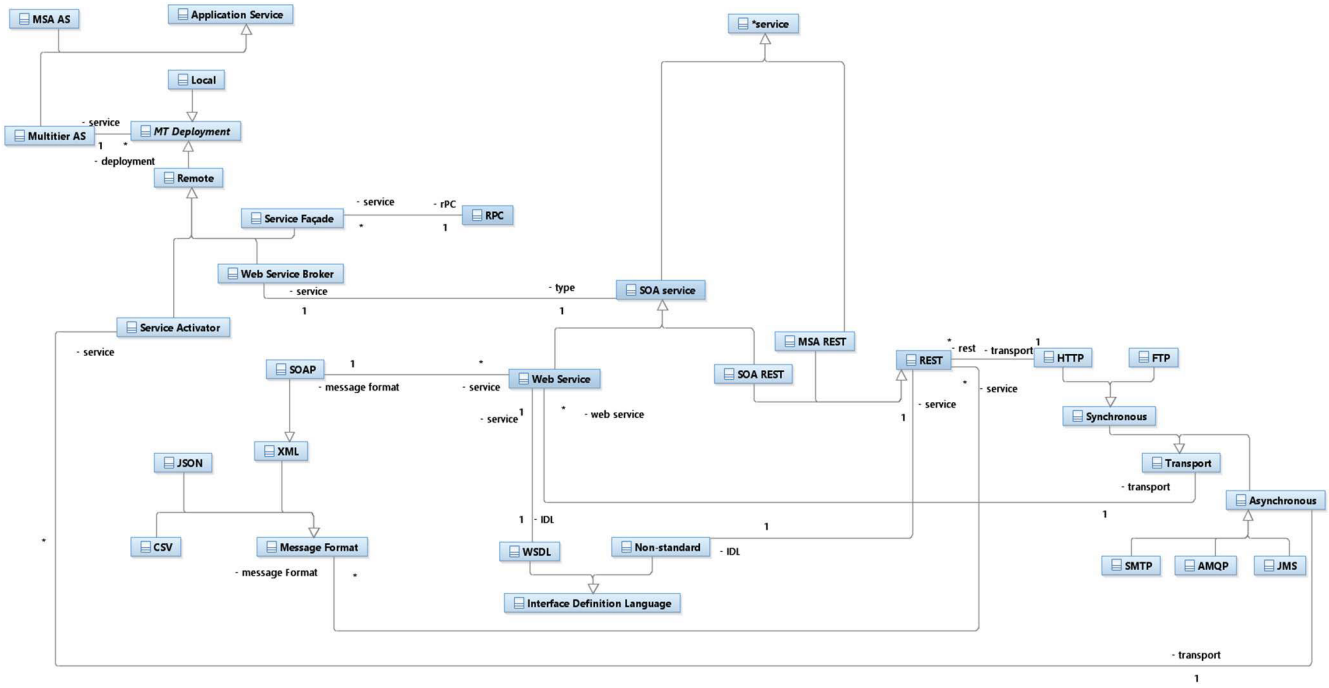


FIGURE 6. MOF metamodel for SOA services.

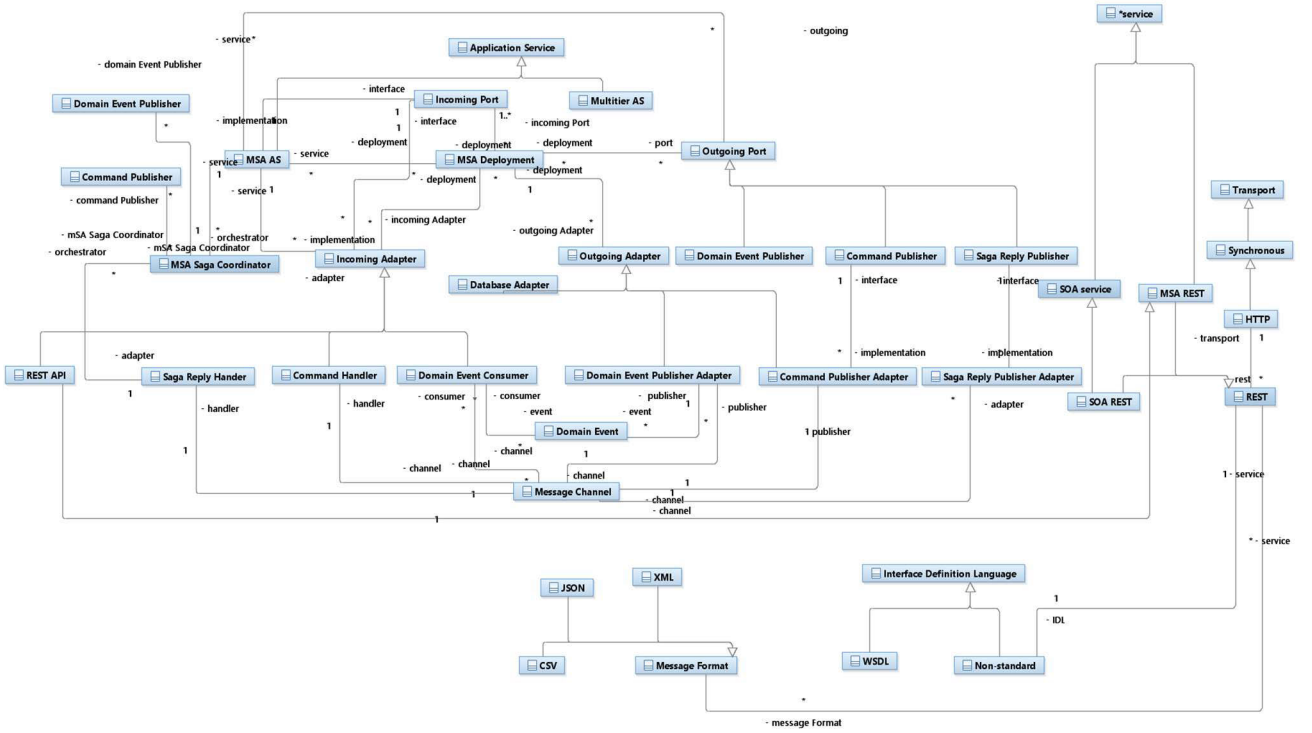


FIGURE 7. MOF metamodel for microservices. Command publisher and domain event publisher are replicated for the sake of simplicity.

these messages are outgoing ports and adapters. Receivers of these messages are incoming ports and adapters. Thus, the application services/application components are not at the same time incoming and outgoing elements, but the concept of primary/secondary is still blurred because elements responsible for dealing with the events launched by the

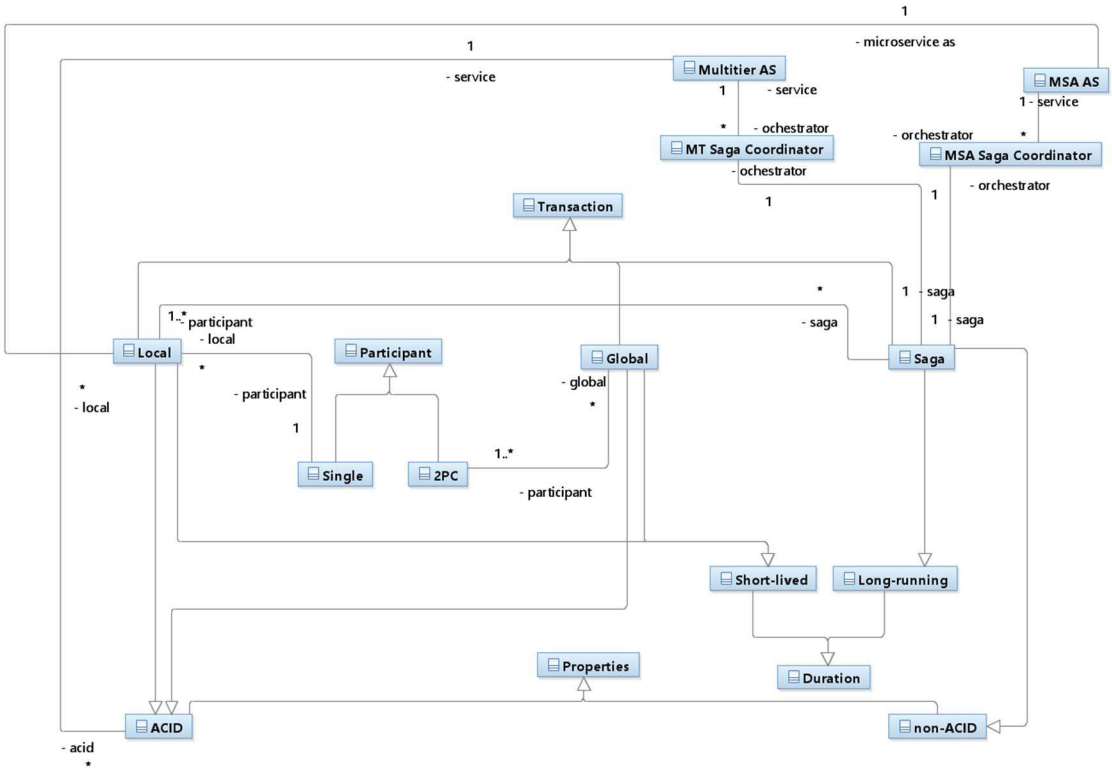


FIGURE 8. MOF metamodel for *service transactions. For the sake of simplicity, only the application services that are exposed as *services appear in the metamodel.

applications are considered incoming elements, although the application is not a primary actor itself.

In any case, apart from finicky technical questions about whether communication between services questions the concept of incoming/outgoing port, may be somewhat excessive calling the hexagonal architecture an architecture. Multitier and microservices architecture catalogues define dozens of patterns that describe information processing in detail, while the hexagonal architecture described by Cockburn is a simple mechanism that basically isolates classes with interfaces and conceives of three types of classes: those that receive data (incoming adapters), those that process them (services) and those that send data abroad by a mechanism other than the reception one (outgoing adapters). To a certain extent, these are three tiers similar to presentation, business and integration (extending the scope of multitier tiers), but without the level of detail of dozens of design patterns. Figure 5 defines this simple mapping between both architectures.

D. MOF METAMODELS FOR SOA SERVICES, MICROSERVICES AND *SERVICES

The MOF metamodel of Figure 1 focuses only on the important elements of the multitier architecture that can be mapped to the hexagonal architecture. Figure 6 expands this metamodel to define a metamodel for SOA services, or *multitier services*. This new metamodel includes the elements that distinguish between web services and REST services [55]: message format, the presence of an *Interface Definition Language* (IDL) and the transport.

According to Figure 6, SOA services can be web services or REST services. Web services are those that use SOAP as message format, WSDL as IDL, and almost any transport. REST services are those that have neither a specific message format nor IDL, and rely on HTTP for their transport [55], [56].

Thus, SOA can be considered a type of multitier architecture where application services are exposed using the web service broker pattern [1]. Indeed, multitier architecture is service-oriented regardless of how the application services are exposed [1]: (i) directly to the web tier controller's *commands* [53] that are in the same physical tier as the business tier; (ii) wrapped in session façades [50], which allow RPC access to them; (iii) wrapped in web service brokers, which allow access to them via SOAP or REST services [56]; or (iv) wrapped in service activators [50], which allow asynchronous message-based access to them.

Some consider web and REST services to be different, but since enterprise buses make automatic conversions among them [57], this paper does not distinguish them and simply calls them SOA services or web services. This is a mere simplification to make the writing of this paper easier and does not attempt to determine whether or not SOAP and REST services should be considered equivalent [1].

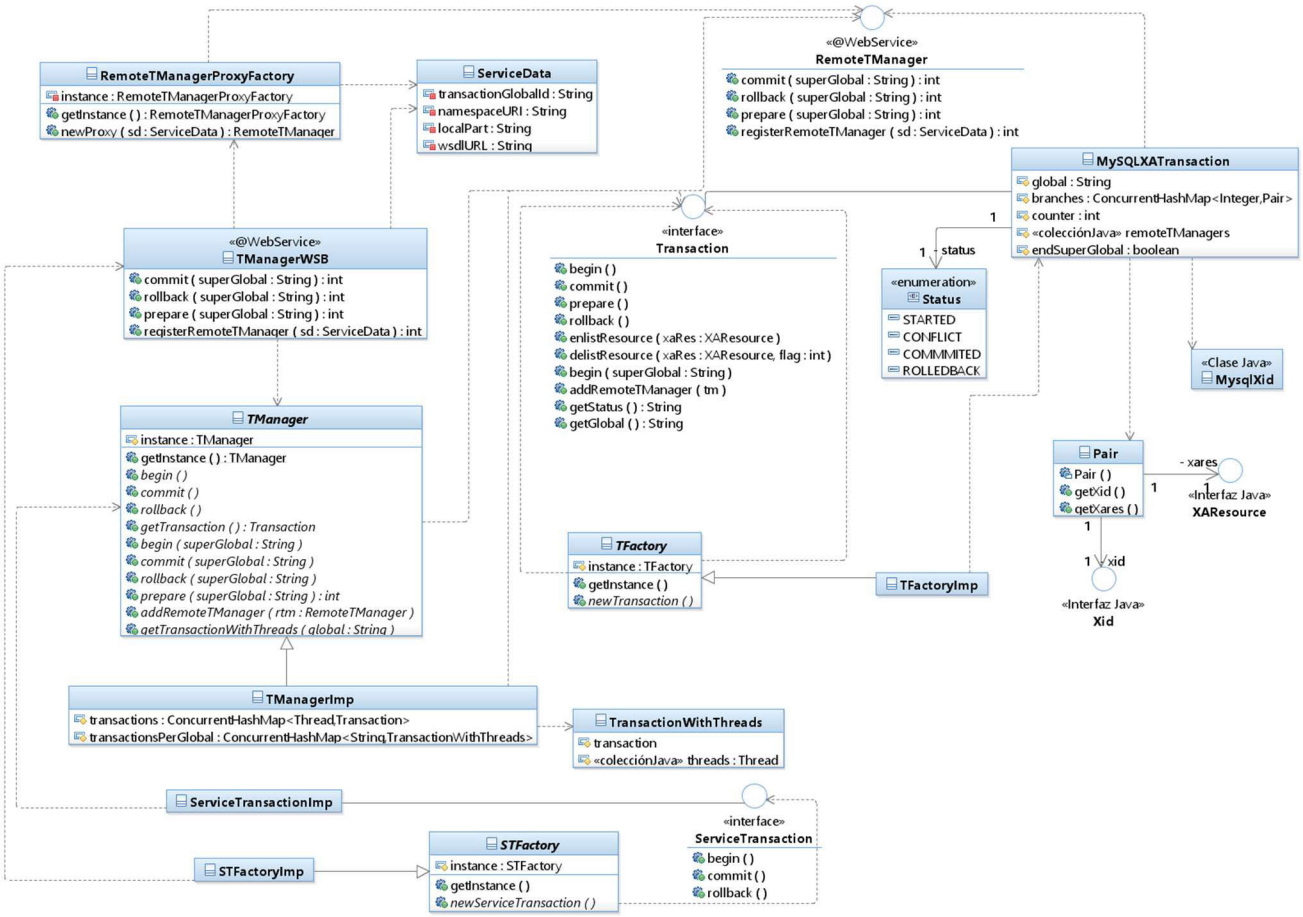


FIGURE 9. Transaction manager for SOA services.

Therefore, hereafter this paper refers to SOAP services and SOA REST services as SOA services.

The hexagonal architecture was defined before the advent of microservices and provides a detailed definition of its components. Therefore, the MOF metamodel for the hexagonal architecture of Figure 2 does not include information about microservices. Figure 7 defines the main elements of microservice architecture [30] and their relationships with message format, IDL and transport. In this case, microservices are the application components of the hexagonal architecture of Figure 2. REST API is the main incoming adapter that communicates external clients with microservices. Although microservice can implement sagas in a cooperative way (*choreography*), the presence of a director that organizes the participants (*orchestration*) is advised due to its advantages [30], [58], and this is the adopted mechanism with the name *saga coordinator*. In microservice architecture, asynchronous interservice communication is preferred to improve system availability. Saga coordinators communicate with services, which perform the operations requested by sagas, through command publisher outgoing ports and their command publisher outgoing adapter. Command handlers are the incoming adapters that process the commands and invoke the services. In addition, sagas use saga reply handlers to receive responses to their commands, which are sent by microservices using saga reply publishers and adapters. This paper does not consider saga reply handlers as incoming adapters, as they are used by saga coordinators and not by general-purpose microservices.

Concurrent data manipulations generate data inconsistencies, which are detected using *domain events* [30] that inform of the events related to data changes. Therefore, it is necessary to have an outgoing port that acts as a domain event publisher, a domain event publisher outgoing adapter to send the events to a message channel, and its counterpart domain event consumer, which plays the role of incoming adapter.

All these ports and adapters are necessary due to the application of the pattern *single database per service* [59], as described in Section V. Although Amazon AWS considers this pattern optional for microservices [60], microservices without a single database per service become something very similar to SOA REST services (see discussion in Section VI-B).

Note that, as defined in Figures 6 and 7, **services* arise as the generalization of SOA services and microservices.

The metamodel of Figure 7 has been inspired by the pattern catalogue provided by Richardson [30]. If public cloud providers in 2024 are considered, AWS holds 31% of the

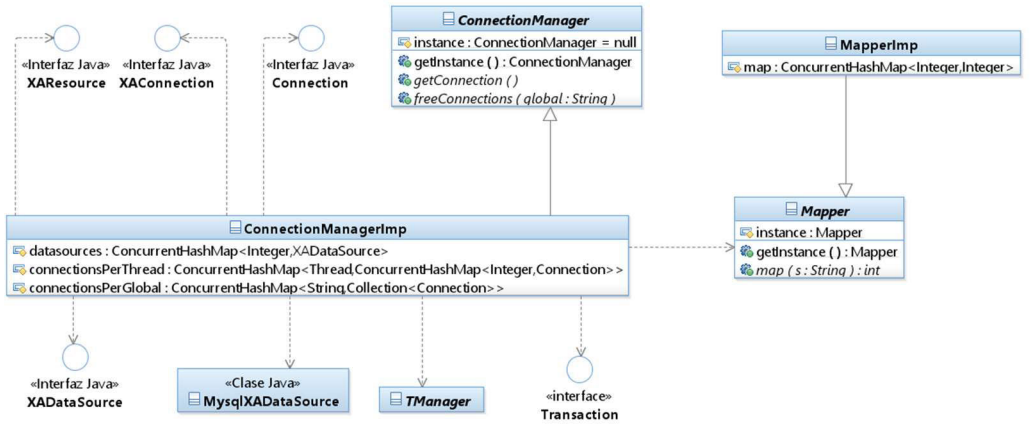


FIGURE 10. Connection manager for SOA services.

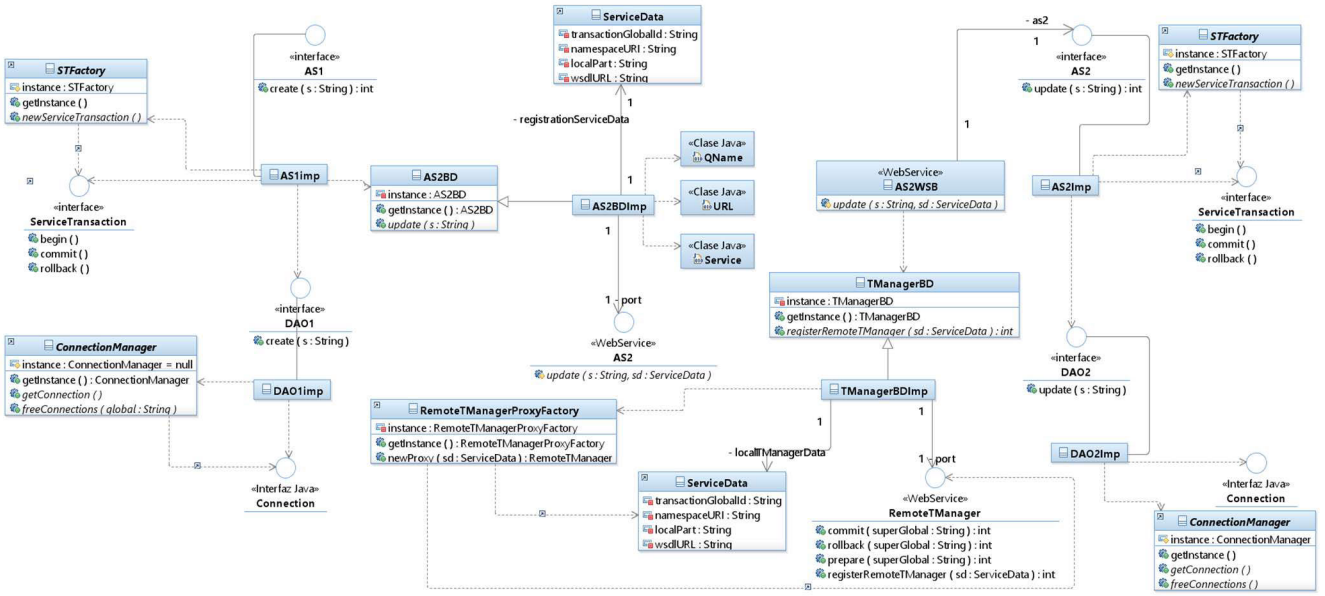


FIGURE 11. Use of the proposed framework by application services and DAOs. `AS1Imp` calls the remote service `using` local `AS2BD`.

market share, Microsoft Azure 24%, and Google Cloud another 11% [61]. These three providers have published different catalogues of MSA patterns. Although each provider has its own pattern catalogue, all of them share a common view on the use of *Domain-Driven Development* (DDD) for domain analysis and its translation into databases [62], [63], [64]. Indeed, the patterns defined by Amazon, Google, Microsoft, and Richardson are based on the same principles and are quite similar. Thus, Richardson's catalogue has been chosen for inspiring the metamodel defined in Figure 7 because it is a provider-agnostic catalogue that gathers the main patterns defined by cloud providers.

E. MOF METAMODELS FOR *SERVICE TRANSACTIONS

My previous paper provides a detailed description and classification of service transactions [1]. Figure 8 summarizes the main concepts regarding service transactions in a MOF metamodel. According to this metamodel, transactions can be local, global or sagas. This metamodel includes a classification considering *properties* (ACID or non-ACID), *duration* (short-lived and long-running) and number of *participants* (single or 2PC). The metamodel does not consider non-transactional resources, because although Richardson thinks that non-transactional resources can be included in transactions using the *transactional messaging* pattern [30], this pattern only guarantees a transactional recording in a relational database of some data and some events, but this does not guarantee that such an event will be processed as a message (or whatever behavior is expected). Figure 8 also includes multitier and microservice application services. According to this metamodel, multitier application services (ASs) can participate in local, global and sagas, while microservice ASs can only participate in local and

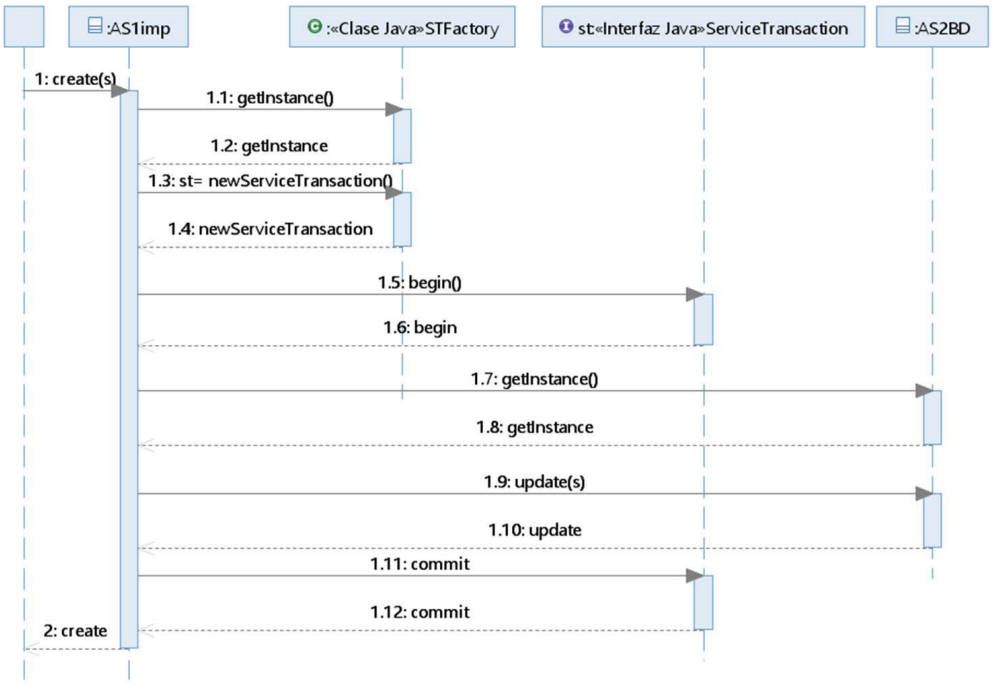


FIGURE 12. Starting the global transaction in the original application service `AS1Imp` and invoking remote service using `AS2BD`.

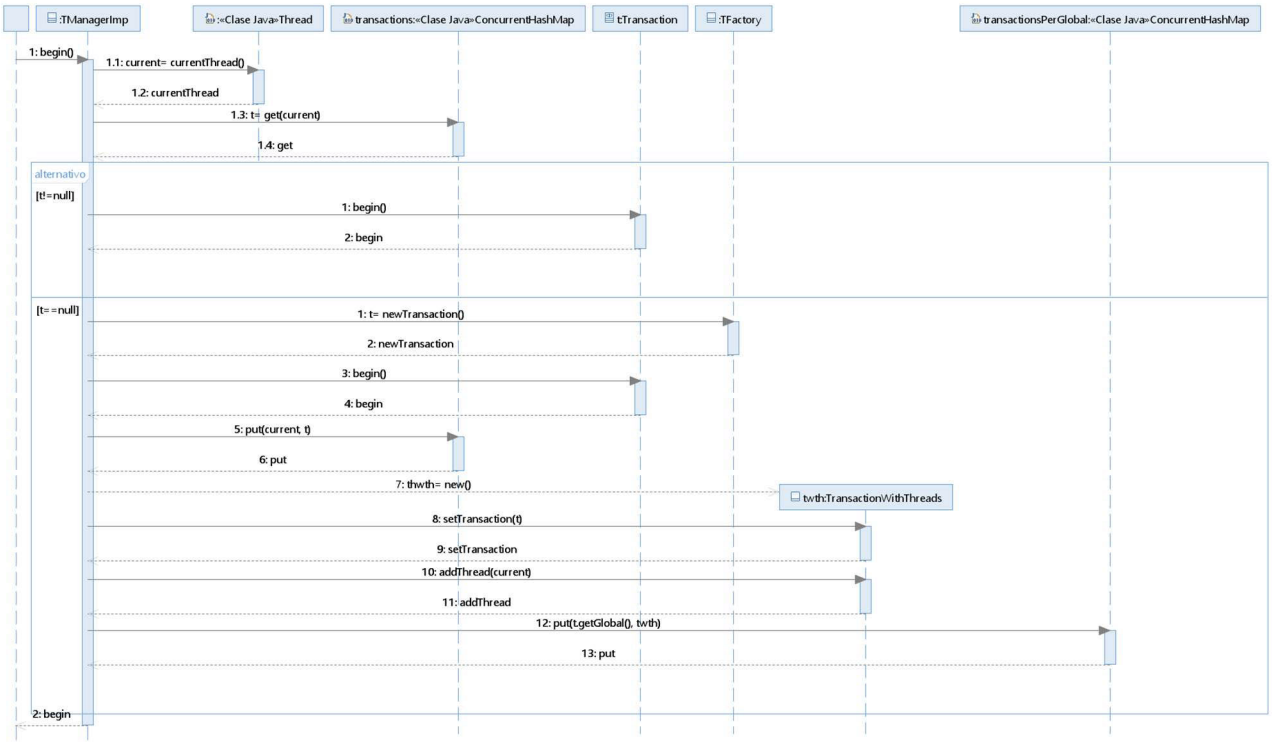


FIGURE 13. Starting a transaction in the proposed transaction manager of Figure 9 without pre-existent global context, e.g. started by `AS1Imp`.

sagas. Note that, unlike microservice ASs, which consider saga coordinators part of the architecture itself, multitier AS does not consider saga coordinators as part of the architecture itself. This is because the microservice architecture needs saga coordinators for any transaction that involves more than one table, regardless of its duration, whereas multitier only

needs sagas for long-running transactions. In addition, multitier saga coordinators can take advantage of WS-Business Activity implementations.

IV. TRANSACTION MANAGER FOR SOA SERVICES

SOA services rely on application servers or frameworks for their implementation. Therefore, there are many transaction managers for dealing with their transactions, which can be used in the context of an application server or as a standalone product [1].

However, these transaction managers are extremely complex and are not suitable for teaching purposes. This section describes a simple transaction manager for SOA services that illustrates the main elements behind the implementation of global transactions for SOA services.

In any case, this paper does not advocate the definition of ad-hoc transaction managers for SOA application with a DAO-based implementation. This paper recommends using pre-existent transaction managers and a JPA-based implementation. The transaction manager depicted in this section is only for illustrative purposes, as implementations of Model-View-Controller pattern are provided for teaching, although in practice frameworks that implement this pattern are used. Its purpose is to provide a proof of concept that is detailed enough to be fully functional and simple enough to be understandable to those interested in transaction management for *services. It may be complex, but it is much simpler than those included in professional frameworks. However, for a better understanding of this transaction manager, it is advisable to check the two transaction managers for dealing with local and global transactions without remote services described in my previous paper [1]. Although these two transactions managers are not able to deal with global transactions and SOA services, they are simplified versions which can provide an incremental understanding of the complexity behind transaction managers: local transactions for single clients, global transactions for single clients, and finally as the transaction manager defined in this paper depicts, global transactions for remote service clients.

The transaction manager depicted here does not completely adhere to WS-Coordination, but reuses some of its ideas:

- An application service starts a transaction, and the transaction manager fulfils the operation.
- The transaction manager plays the role of coordinator.
- Transaction creation is similar to activation: it creates a transaction with a unique identifier and a transaction context with this identifier and the address of the transaction manager's registration service.
- When the application service that started the transaction invokes other web services, the transaction context is included.
- The invoked services also have a transaction manager that uses the address of the registration service to register itself as a participant in the transaction.
- The transaction in each machine takes care of the branches in the machine in terms of the local resource managers participating in the transaction and the remote coordinators that also participate in the transaction.
- Participant transaction managers have to agree on the protocol they use. For example, a 2PC protocol forces the presence of a prepare/commit-request operation for the agreement of the participants. In the transaction manager defined in this paper, only the 2PC protocol is supported.
- This transaction manager does not support long-running transactions, but can participate in sagas, as sagas do not require agreement between participants, since it is the coordinator (for orchestrated sagas) who takes care of this job.

The core elements of the solution are depicted in Figures 9 and 10. Figures 11 and 12 depict the use of these classes by application services.

The transaction manager is responsible for tracking transactions per thread and transactions per global identifier. In the latter case, it is useful that the transaction keeps track of all the threads where the transaction can be present (note that the same application may receive different requests in the same transaction), although it is only possible to have one active transaction per thread. The transaction manager has a web service broker (that implements the `RemoteTManager` interface) responsible for dealing with the remote request of protocol-dependent transactional operations (commit, rollback, prepare) as well as with the registration of remote participants/transaction manager invoked by the application service that originated the transaction. The different roles played by the transaction manager could be separated into different interfaces/web service brokers, but in Figure 9 they are unified for the sake of simplicity. Note that the JAX-WS proxies for accessing the remote transaction managers, registered in a transaction (Figure 9) or received in the context of a global transaction (Figure 11), are built using a factory (`RemoteTManagerProxyFactory`). Figure 13 describes the start of a transaction using a transaction from a local application service (i.e. without a pre-existent global context) as the one in Figure 12. Figure 14 describes the start of a transaction as a result of receiving a request for an application service from another remote service that started the transaction (i.e., with a pre-existing global context). In the first case (Figure 13), a new transaction is created (or *activated* using WS-Coordination terminology) and started, stored per thread, and stored per global identifier (considering the thread in which it has been created). In the second case (Figure 14), if it is the first time that this service has been called, a similar process to the one depicted in Figure 13 is performed. Otherwise, the new thread is stored in the global transaction, and the transaction is stored per thread. The transaction is also started, which only increments the counter of commits performed in any case.

Figure 15 depicts the request sent by the implementation of the *business delegate* [50] of remote service `AS2 (AS2BDImp)` after being invoked in the context of an application service (Figure 12) that started the transaction in the

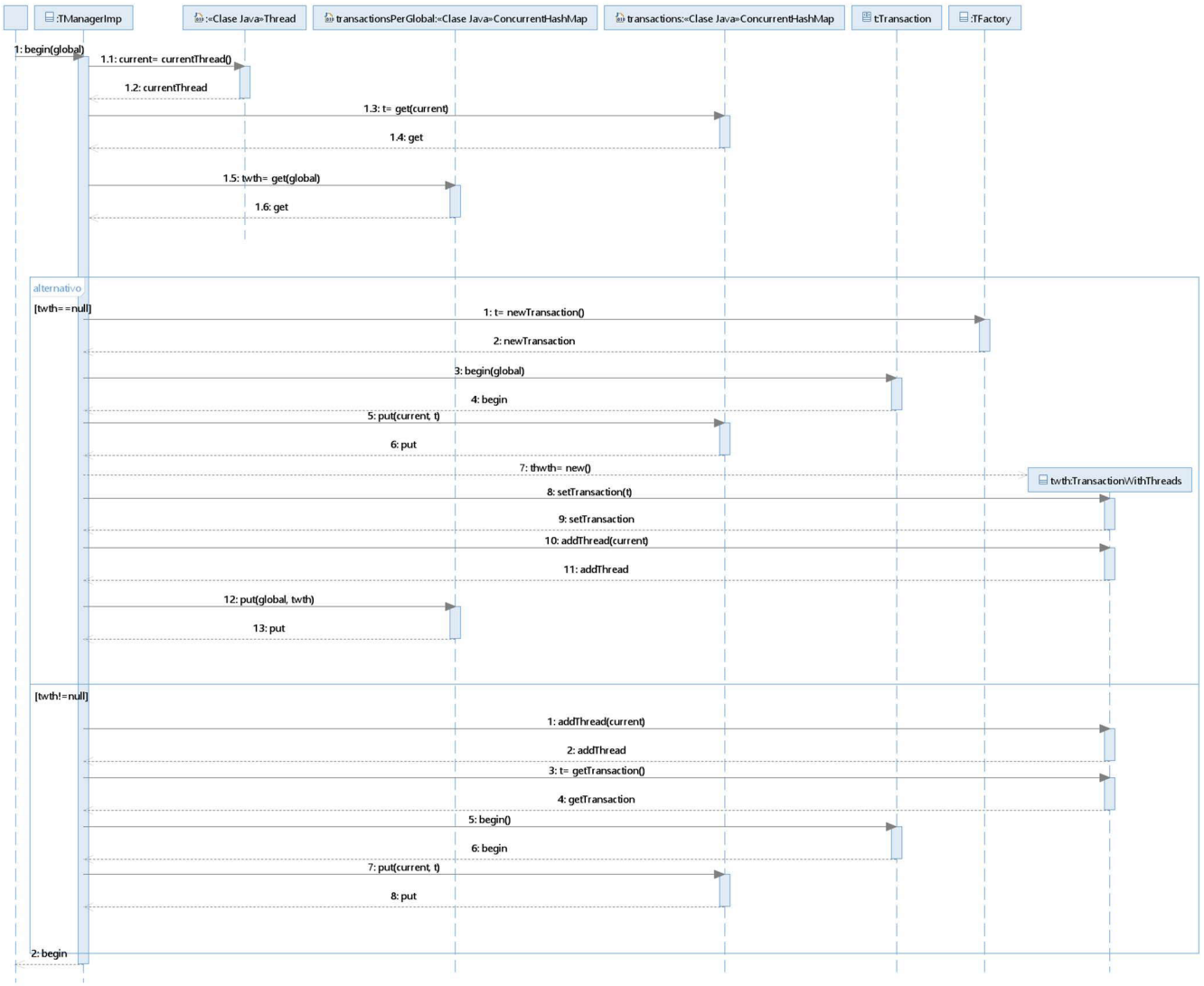


FIGURE 14. Starting a transaction in the proposed transaction manager of Figure 9 with a pre-existent global context, e.g., received by `A2WSB`.

local machine according to Figure 13. Note that this business delegate gets the identifier of the global transaction and creates the `ServiceData` for accessing the registration service of the local transaction manager within this global identifier (i.e., the *context* in WS-Coordination). This data is sent to the requested service, along with the information needed for processing the request. The `namespaceURI` and `localPart` attributes needed for building a `QName` contain the web service `targetNamespace` and the service `name`. Both are the same for every application that wants to use the transaction manager defined in Figure 9. `wsdlURL` contains the URL where the WSDL that defines the transaction manager interface is deployed, and `transactionGlobalId` contains the transaction global identifier.

This request is received by the `AS2` web service broker (`A2WSB`), as Figure 16 depicts. The `AS2` web service broker initiates a transaction in the local transaction manager with the global identifier received (according to Figure 14), performs the operation implemented by the application service, and calls the transaction manager business delegate to register the local transaction manager under the global transaction. Figure 17 shows the behavior of this transaction manager business delegate, which holds the data of the local transaction manager: it creates a proxy for the remote registration service according to the data received, sets the value of the global transaction identifier in the local service data (i.e. the context), and uses the proxy to register the local transaction manager. Figure 18 depicts the reception of this request in the transaction manager that holds the initial transaction (i.e. the transaction started in Figure 12) and the registration of the remote transaction manager in the global transaction.

The `Transaction` interface described in Figure 9 is, to some extent, equivalent to the `jakarta.transaction.Transaction` JTA interface. The

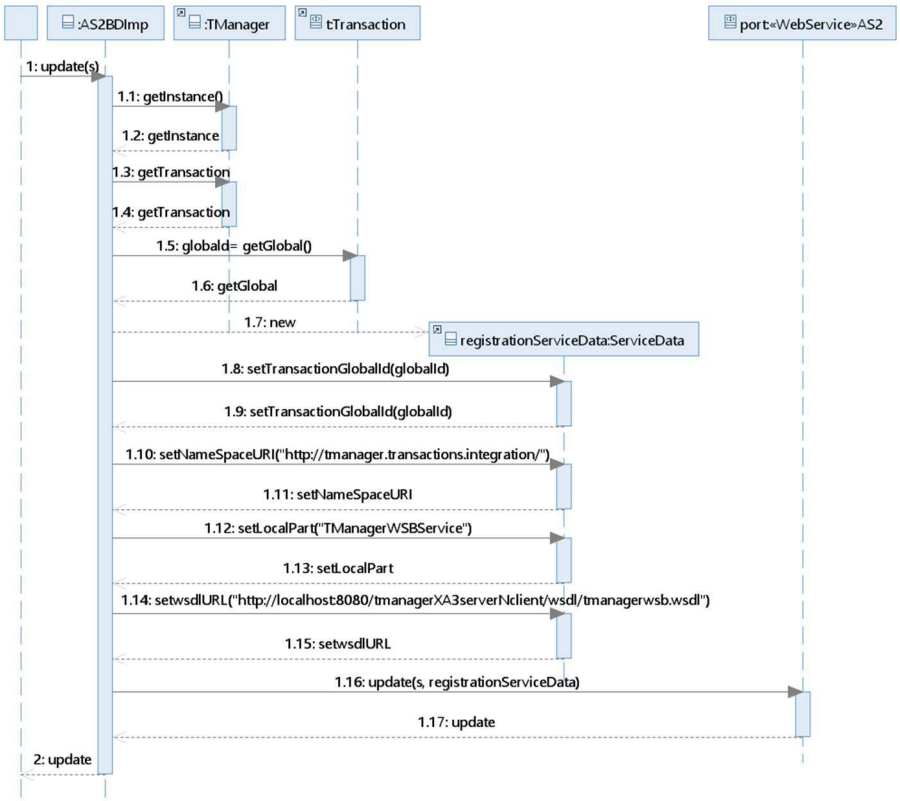


FIGURE 15. The business delegate in the client creates a context and includes it in the request to the remote service.

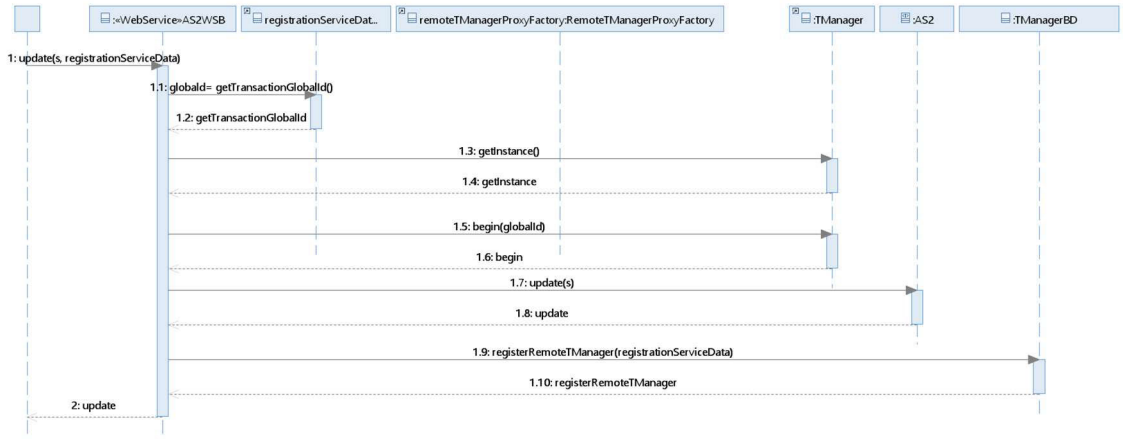


FIGURE 16. The web service broker receives the request, invokes the business logic, and initiates the registration of this participant.

`MySQLXATransaction` class that implements it takes care of the global ID, the branches, the number of begins and commit/rollback actions performed, the status of the transaction, the remote transaction managers that participate in the transactions, and whether a global transaction can commit.

As in the case of simpler transaction managers for XA transactions defined in [1], the branches are `Pairs` of (`Xid`, `XAResource`) objects indexed by the `XAResource` hash code. This information is needed because, in the two-phase commit, XAResources need the branch id (`Xid`) in order to interact with the resource managers. Figure 19 shows the enlistment of a resource in a transaction, and Figure 20 shows the commit of the transaction according to the two-phase commit protocol. Figure 21 depicts the preparation of the local resource managers. Figure 22 depicts the commit of the

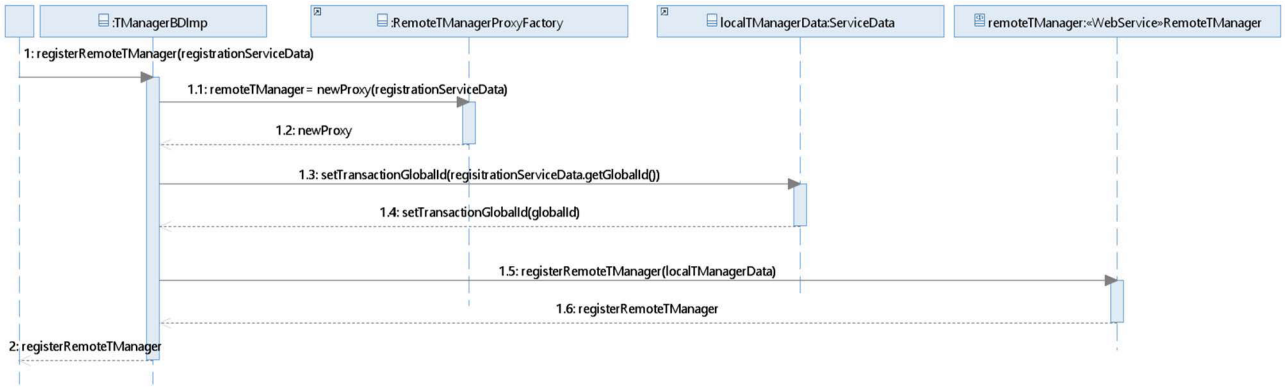


FIGURE 17. In the invoked machine, the participant initiates its registration in the transaction manager from the invoker service.



FIGURE 18. The initial coordinator registers the invoked participant in the context of the global transaction.

local resource managers. Figure 23 (a) depicts the preparation of the remote transaction managers and (b) depicts the commit of the remote transaction managers.

The `ServiceTransaction` interface described in Figure 9 is, to some extent, equivalent to the `jakarta.transaction.UserTransaction` JTA interface, and it is a simple adapter that hides the existence of the transaction manager from application services. Thus, for example, the code of `ServiceTransactionImp::begin()` is simply `TManager.getInstance(). begin ().`

Connection management for this transaction manager has the same caveats that the connection manager that manages the connections for XA transactions defined in [1]. Taking into account these caveats, the management of connections made in Figure 10 is fairly simple. If n RDBMs are going to be used, n configuration files must be provided for the configuration of each `XADataSource`. Therefore, in this approach, each RDBMS is bound to a specific `XADataSource`. Each RDBMS/`XADataSource` has a unique ID, provided by the developer. Thus, if there are three RDMBSs, there will be three IDs: 1, 2 and 3 (the order is not important). An additional configuration file that binds DAO classes with RDMBS/`XADataSource` IDs is provided. The singleton `ConnectionManager/ ConnectionManagerImp` holds a `ConcurrentHashMap` that indexes each `XADataSource` by its ID (`dataSources`). These data sources are configured in the constructor of the class `ConnectionManagerImp`. This singleton also holds the JDBC `Connections` established in a transaction. Because in this solution there can only be an active transaction per thread, the `ConnectionManagerImp` indexes connections by execution thread (note that in Figure 20 the commit frees the connections bound to a transaction). These `Connections` have to be bound to their RDBMS/`XADatasource` ID. Therefore, bound to a thread, they are held in a `ConcurrentHashMap` that

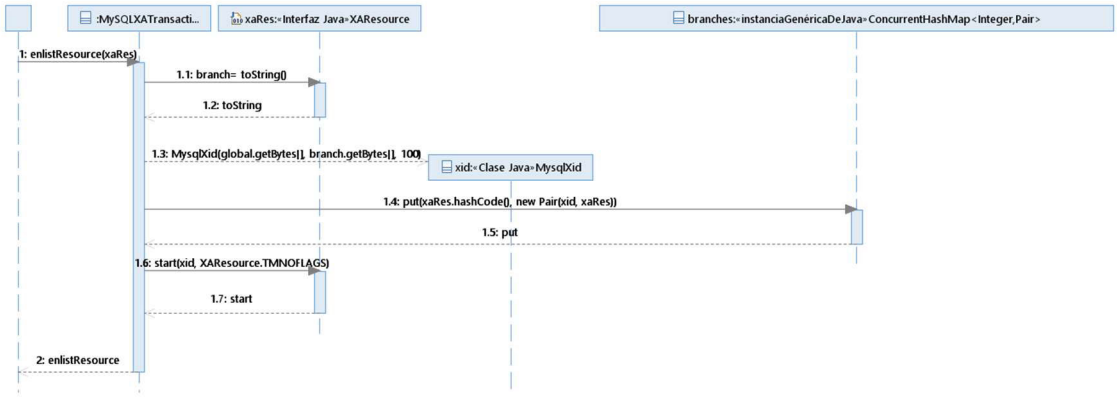


FIGURE 19. Transaction enlistment in the class that implements the `transaction` interface of Figure 9.

indexes them by this ID (`connectionsPerThread`). Thus, this solution only allows one active connection per RDBMS/`XADataSource` in a thread. Because each transaction needs to participate in the global transaction, it is necessary to take care of the connections bound to the global ID in the `ConcurrentHashMap connectionsPerGlobal`.

Each time a `Connection` is required by a DAO, a Mapper class maps the class of the DAO with the RDMBs/`XADatasource` ID, and the connection manager checks if there is a connection bound to this ID in the thread. If there are none, it gets the `XADatasource` bound to the ID and generates a `XAConnection`, which is used to generate an `XAResource` and a `Connection`. This mapping is driven by a configuration file that maps the DAO classes with the datasource's IDs. The `XAResource` is enlisted in the transaction bound to the thread and the `Connection` is stored in the connections bound to the thread (indexed by the RMDBs/`XADatasource ID`). Finally, the connection is bound to the global identifier. Figure 24 describes this process. Due to the size of the diagram, the figure has been divided into three parts, (a), (b) and (c), where interactions in part (a) precede those in part (b) and those in part (b) precede those in part (c). Note that the diagram coincides with the case for the transaction manager defined in [1], except for part (c), where the new connection is bound to the global transaction identifier.

V. *SERVICE TRANSACTION MANAGEMENT

Previous sections focused on the concepts related to *services and transaction managers for SOA services. Transaction managers for SOA services (or even the simpler transaction manager defined in [1]) can also handle the local transaction management performed in microservices, so there is no need to design a specific transaction manager for microservices. This section gathers the knowledge defined in previous sections and illustrates the differences in transaction management between SOA services and microservices.

The database per service pattern, as well as the inability of microservices to participate in global transactions, force important differences between microservices application services and multitier application services. This section includes five use cases that help to illustrate these differences. In all cases, SOA services and microservices are implemented in Java, and use JPA with optimistic concurrency management for data storage [1].

Questions such as consistency, scalability and fault tolerance are some of the main advantages claimed by proponents of microservices, being Amazon and Netflix the paramount applications that encompass all these advantages [30]. Regarding these issues, Zhou et al. [65] concluded that "many advantages, such as high scalability and extensibility, may be achieved by decomposing a monolithic system into independent microservices with smaller granularity. However, it does not mean that the smaller the microservices are, the better. Decreasing the size of microservices may conversely increase the number of communications among distributed services and thus impact the performance of the application or cause the extra complexity of joint debugging and the excessive communication among teams". This section compares SOA services and microservices from a transactional point of view, something that has been traditionally obviated in literature. Section VI-B also draws some conclusions regarding consistency, scalability and fault tolerance based on the examples provided in this section.

For the sake of conciseness, interservice security has been obviated in the examples but it is a critical concern that must be considered [6], [7], [66]. Section VI-D addresses some security issues.

A. LOCAL. ONE TABLE INVOLVED

Let's consider the use case *employee deregistration*: the application receives the employee's identifier, checks whether it is active and deactivates them. This use case does not involve the validation of anything outside the employee table (if the employee belongs to a department or participates in several projects, it is irrelevant for this use case). Figure 25 depicts this use case.

Regardless of specific use cases, SOA services consider a network of JPA entities equivalent to the domain

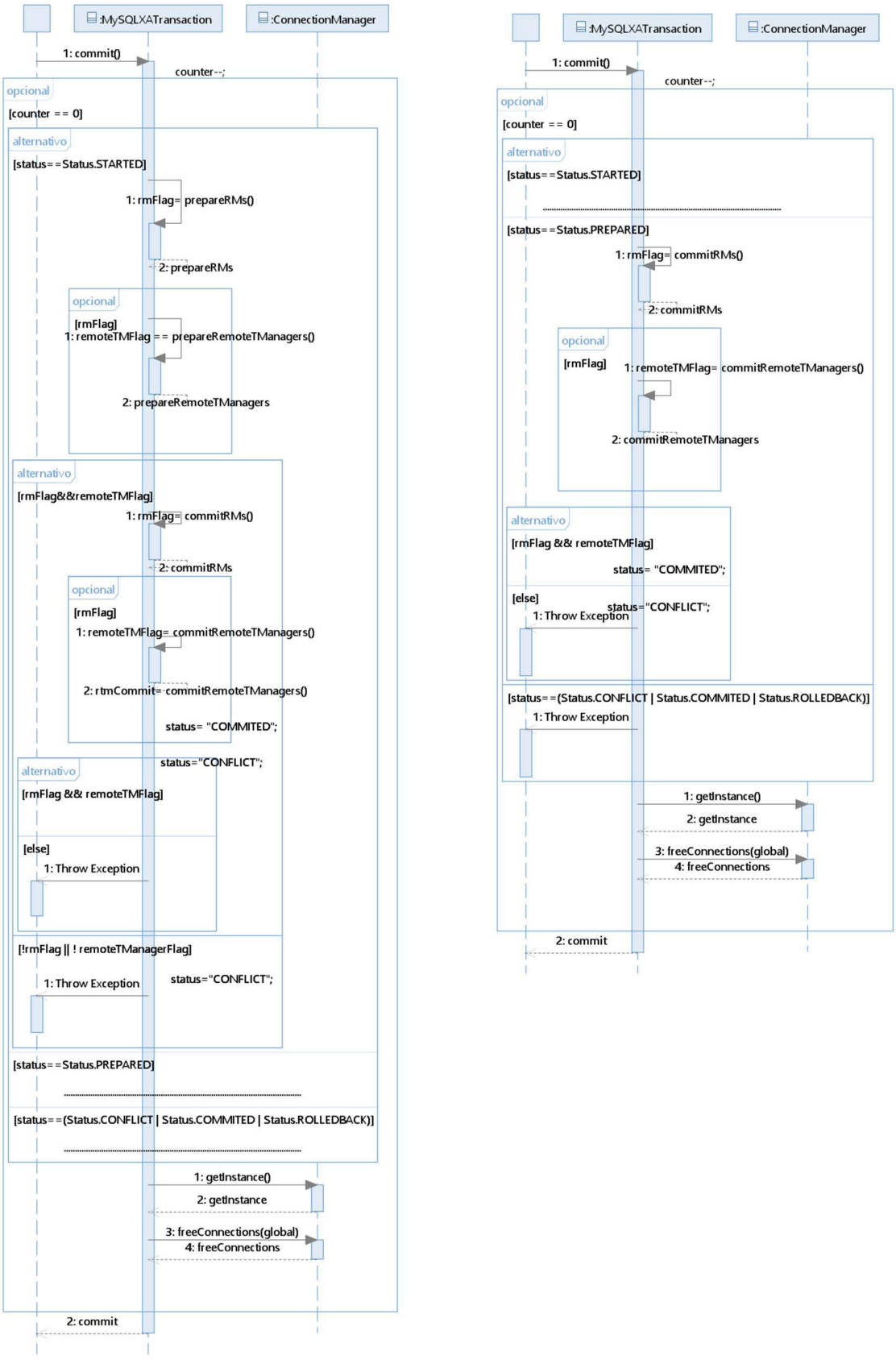


FIGURE 20. Transaction commit in the class that implements the transaction interface of Figure 9. (a) Status started. (b) Rest of status.

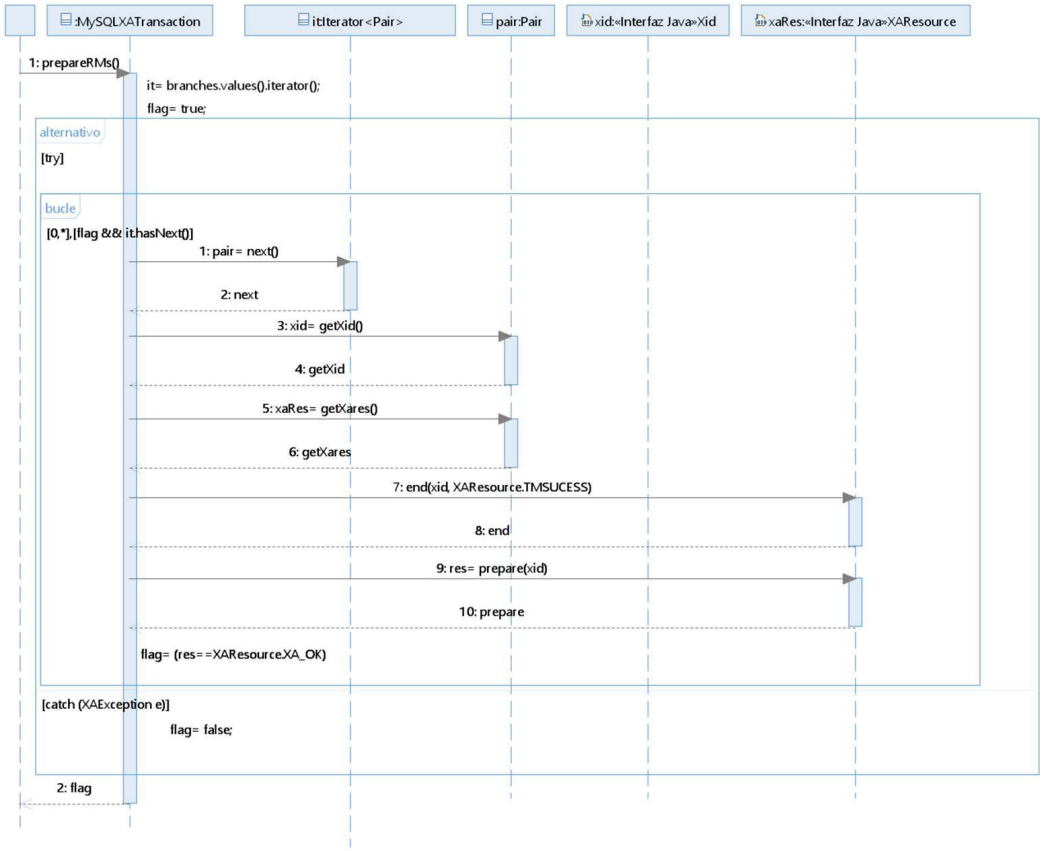


FIGURE 21. Preparation of local resource managers.

relationships, while microservices only consider a single employee table that keeps track of its department using a department identifier.

Figure 26 depicts the JPA entities and application services for the SOA application service.

Figure 27 depicts how the SOA application service implements the use case. No optimistic locking is necessary because the employee is going to be modified, and that forces version control when committing.

Figure 28 depicts the microservice entity, the microservice application service and the classes responsible for publishing domain events (`EmployeeDEP`) and for consuming domain events (`EmployeeDEC`).

Figure 29 depicts how the MSA application service implements the use case. It is very similar to the SOA case, but the MSA application service sends a domain event to make the change in the state of the employee known to those services concerned with this change.

Although application servers and frameworks allow declarative transaction management, this example uses explicit/programmatic transaction management [1] to illustrate how transactions are started and ended in the context of application services.

The next examples use declarative transaction management for the sake of conciseness. Although declarative transaction management requires annotations, which vary according to the framework (e.g., EJB vs JAX-WS) or the container (e.g., application server vs Spring), the examples use EJB transactional attributes, being aware that their mapping (and compatibility) to JAX-WS/WS-Atomic Transaction depends on the specific application server/framework configuration mechanisms [67]. For the sake of simplicity, the injection of JPA entity managers in the application services is obviated.

B. LOCAL. TWO TABLES INVOLVED, ONLY ONE (APPARENTLY) MODIFIED

Let's now consider the use case *employee registration*: the application receives the employee's data, including its department identifier, checks that the department exists and is active, and creates the employee. For the sake of simplicity, let us suppose that no check about employee pre-existence is made. Figure 30 depicts the activity diagram for this use case.

Figure 31 shows the SOA classes involved in the use case implementation. Figure 32 depicts the interaction diagram that implements the use case in the SOA version. Since the employee service can access both employee and department tables, the same operation defines an optimistic force increment lock on the department under validation and creates the employee. This lock detects changes in

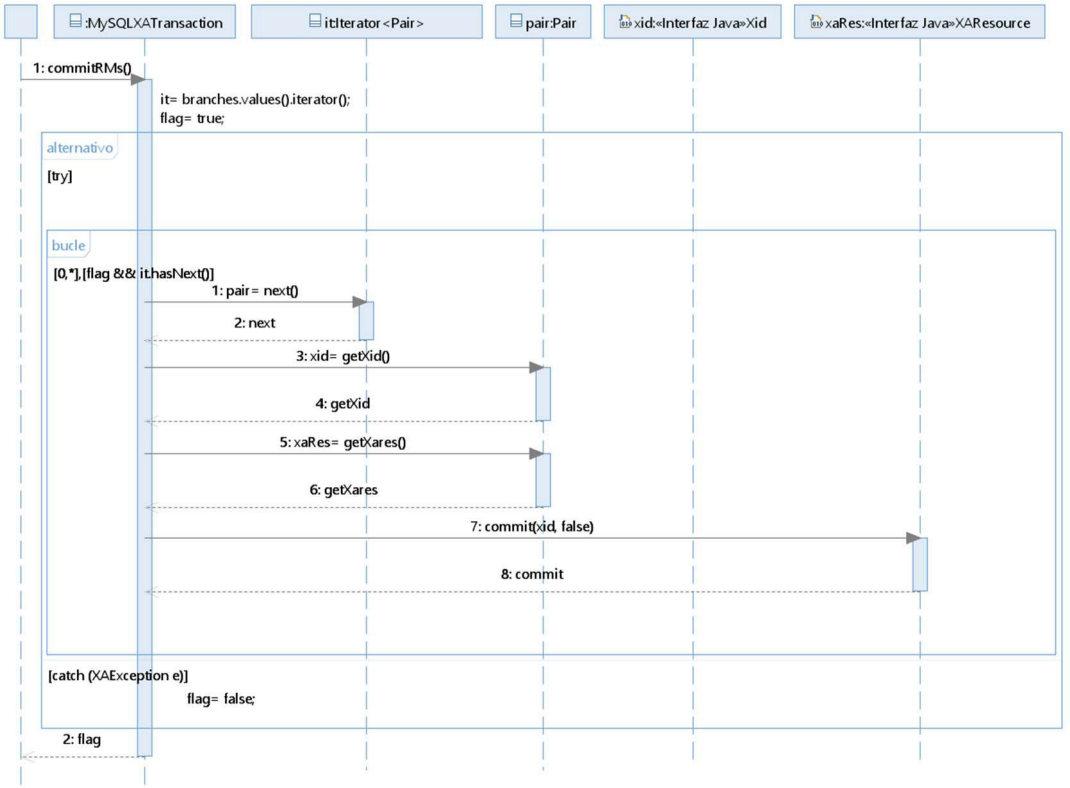


FIGURE 22. Commit of local resource managers.

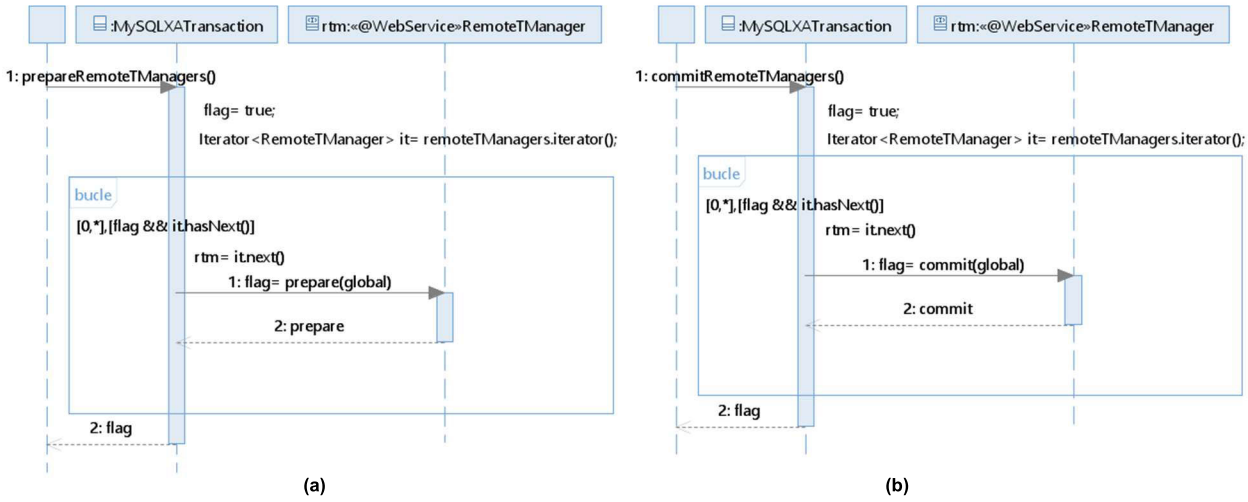


FIGURE 23. (a) Preparation or remote transaction managers. (b) Commit of remote transaction managers.

the department after its validation at the time of commit. In addition, it increases the version number of the department, warning other transactions that could have made some validation on the department for calculating its payroll, for example.

Due to the database per service pattern, the microservice implementation forces each application service to access only its related table, as shown in Figure 33. This figure shows the class diagram of the microservices necessary for the implementation of the use case. Note that both employee and department applications services have domain event publishers (DEPs) and domain event consumers (DECs) for asynchronously sending the domain events that can be of interest to the services concerned (e.g. department

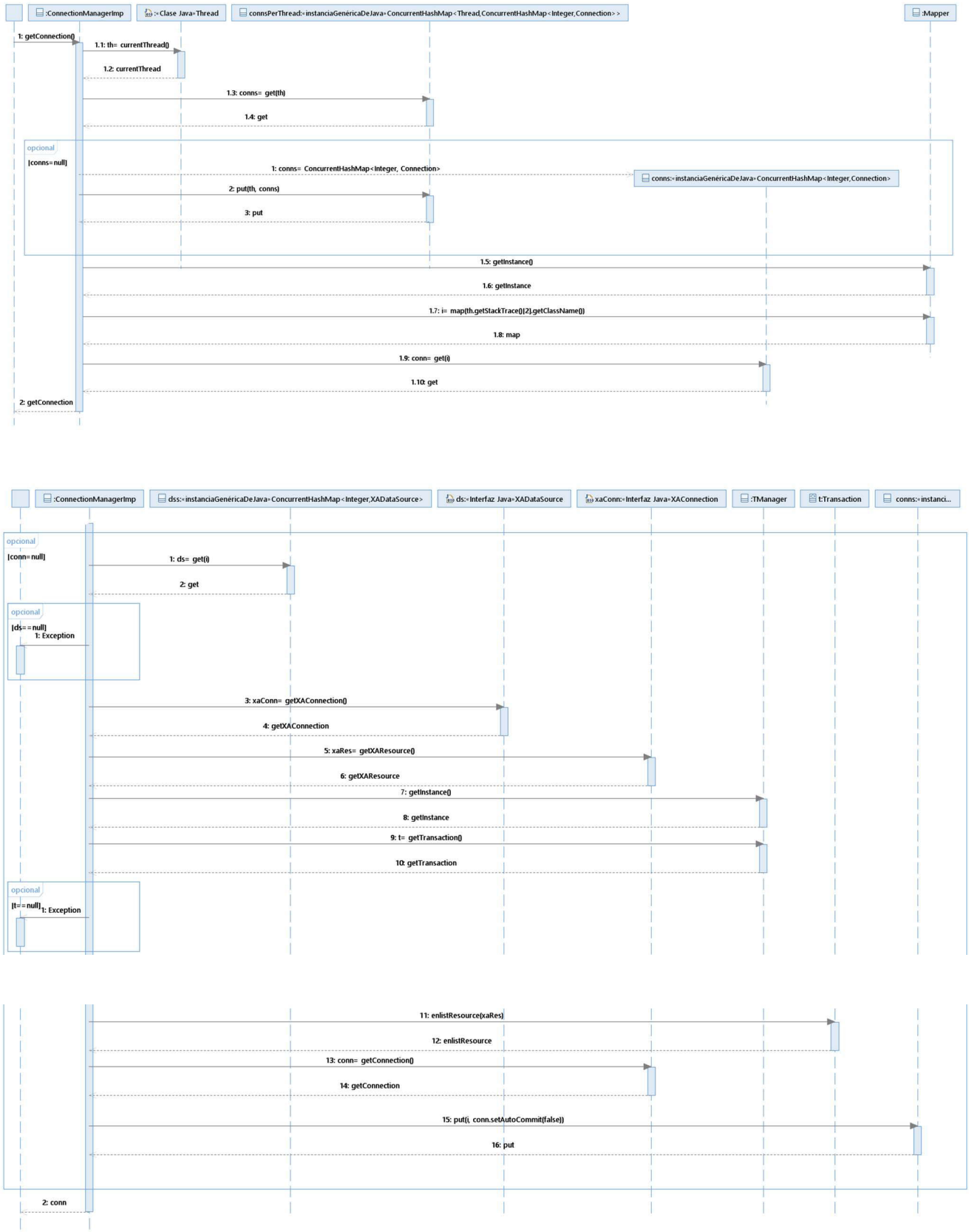


FIGURE 24. **a.** `Connection` **management in the** `ConnectionManagerImp` **class of Figure 10 (first part). b.** `Connection` **management in the** `ConnectionManagerImp` **class of Figure 10 (second part). c.** `Connection` **management in the** `ConnectionManagerImp` **class of Figure 10 (third part).**

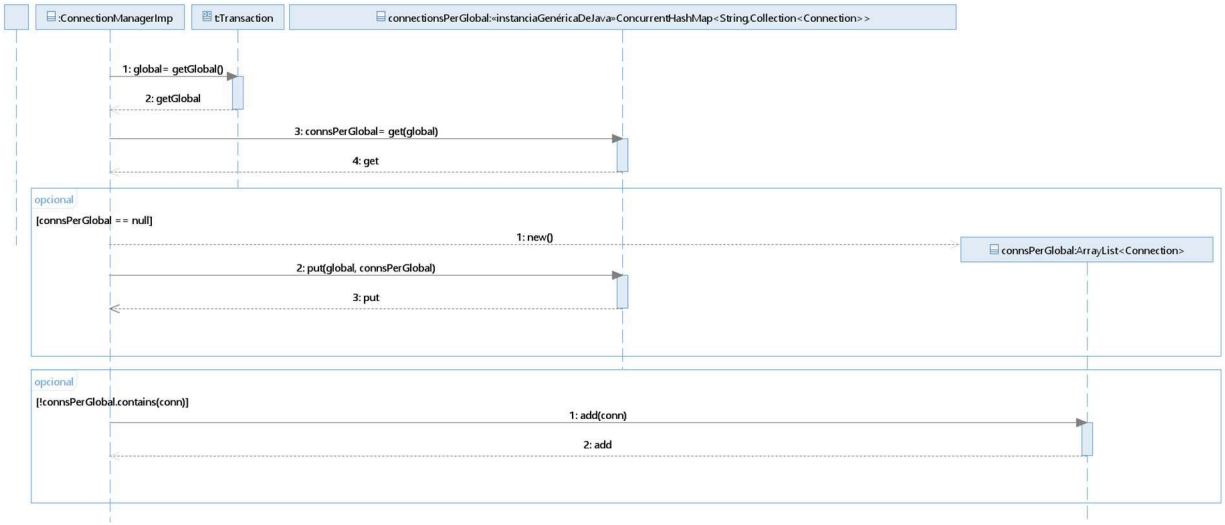


FIGURE 24. ***(Continued.)*** **a.** `Connection` **management in the** `ConnectionManagerImp` **class of Figure 10 (first part). b.** `Connection` **management in the** `ConnectionManagerImp` **class of Figure 10 (second part). c.** `Connection` **management in the** `ConnectionManagerImp` **class of Figure 10 (third part).**

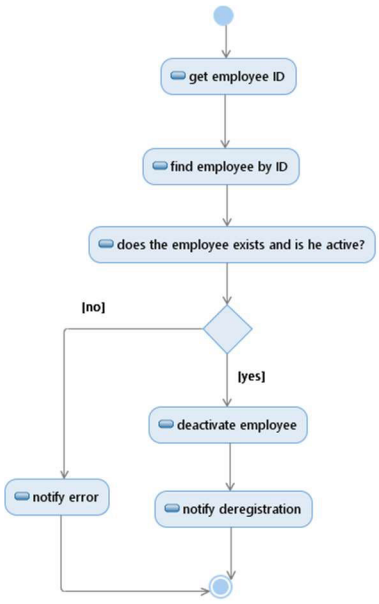


FIGURE 25. **Activity diagram for the use case of employee deregistration.**

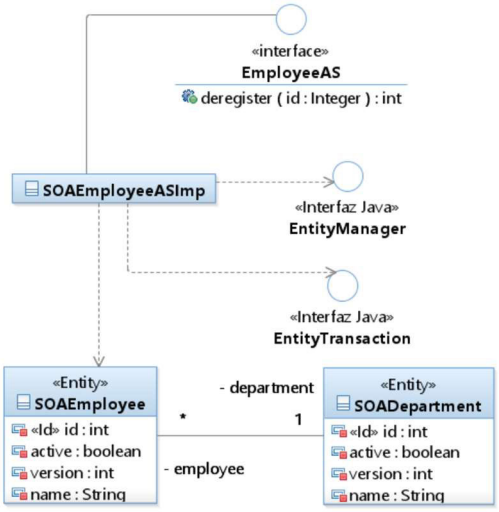


FIGURE 26. **Class diagram for the SOA solution of employee deregistration.**

deregistering or employee registering). In the example, JMS has been chosen because it warrants a transactional behavior, making it unnecessary to simulate it with the pattern of transactional messaging. JMS topics can be used for message publishing because several services can be interested in receiving the same event.

Before providing an interaction diagram for this use case, let us consider some implementation issues. The employee application service must communicate with the department application service in order to check that the department exists and is active. Another option is to use data replication [30], but this solution is expensive and requires events to keep data consistency between the data and its replicas. Going back to interservice communication, microservices promote asynchronous communication among them, although the user receives a partially validated reply (e.g., an employee is created even though its department does not exist) [30]. In order to avoid the complexity of asynchronous communication, which requires a saga coordinator, let us suppose a simpler synchronous communication between services, as illustrated in Table 2. In this scenario, department 17 is active with no active employees at time/step 0. Then, at time 1, two HTTP REST requests collide: somebody wants to register employee 153, who belongs to department 17, and

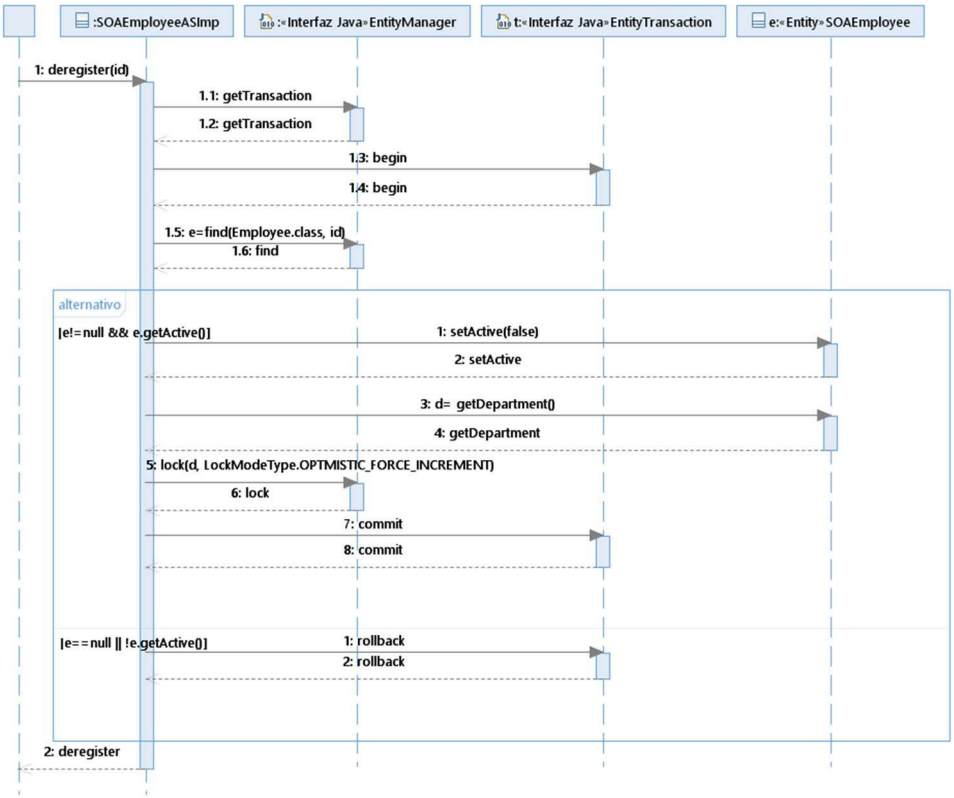


FIGURE 27. Sequence diagram for the SOA solution for the use case of employee deregistration.

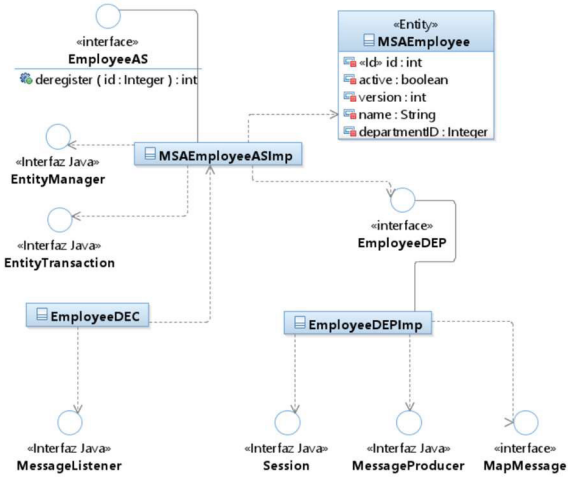


FIGURE 28. Class diagram for the MSA solution of the use case of employee deregistration.

somebody wants to deregister the department 17. At time 2, the process responsible for employee registration checks that department 17 is active, and the process responsible for the department deregistering checks that department 17 is active and has no employees. Both checks are OK, so at time 3, department 17 is marked as inactive. At time 4, a domain event is launched informing that department 17 has been deregistered, whilst employee 153 is registered as belonging to department 17. Thus, at time 5 there is a data inconsistency in the application: employee 153 is active and belongs to department 17, which is inactive. At this time, an event informing of the employee registration is launched. If events launched at time 4 and 5 had not been launched, the data inconsistency would remain. At time 6, both domain events are received, the inconsistency is detected, and both compensating transactions are launched (activate department 17 and deregister employee 153). They are both received and executed at time 7 and domain events informing of the changes are launched (but department is not concerned about employee deregistration and employee is not concerned about department activation). Finally, the data remains consistent at time 8. Note that if the compensating transaction that activates the department is run before the reception of the event that informs about the employee creation, then both the department and the employee would remain active. On the contrary, if the compensating transaction that deregisters the employee is run before the reception of the event that informs about the department deregistration, then both the department and the employee would remain inactive.

Table 2 supposes synchronous inter-process communication at step 2. Asynchronous inter-process communication forces the presence of saga coordinators, one for each process, [30], but if the synchronous validation made by both processes at step 2 is asynchronously performed before the

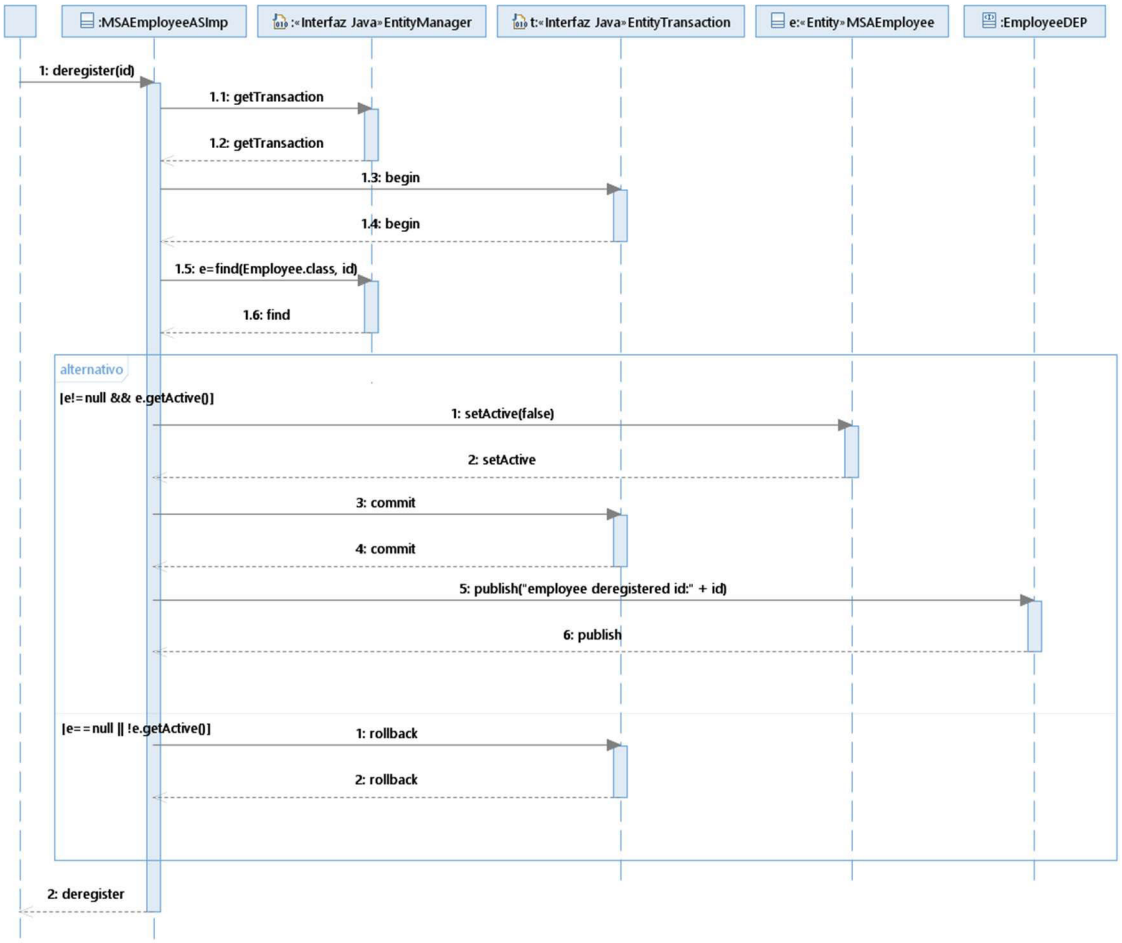


FIGURE 29. Sequence diagram for the MSA solution for the use case of employee deregistration.

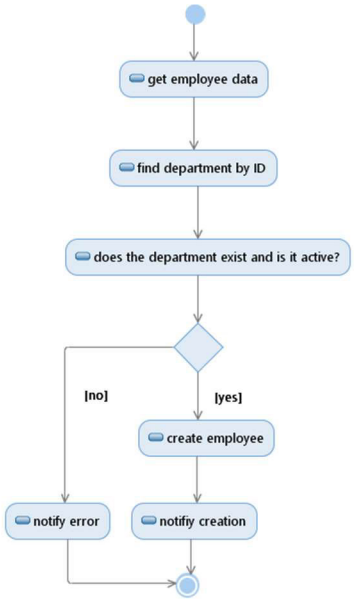


FIGURE 30. Activity diagram for the use case of employee registration.

department is deregistered or the employee is registered, the saga coordinators would continue with the execution of both processes, and the execution trace would be the same. The problem is that the validation of the department in both processes (department deregistering and employee registering) is not integrated into the same transaction that deregisters the department or registers the employee (unlike in Figure 32 for the SOA solution). Therefore, the only way to detect inconsistencies is by launching domain events that force the concerned services (department in the case of employee registration, and employee in the case of department deregistration) to perform further verifications and run compensating transactions, if necessary.

Thus, since validations, either synchronous or asynchronous in the context of sagas, are not trustworthy, why should they be performed? Table 3 shows an alternative code where no validations are made at the time of registering employees or deregistering departments. Now, in Table 3, at step 2, the employee is registered and the department is deregistered, and the casuistry about the reception of events and the execution of compensating transactions is the same as in Table 2, but with a simpler code.

Figure 34 shows the implementation of the use case in the employee applications service following the philosophy of Table 3. For the sake of conciseness, the employee DEP

TABLE 2. Business logic with synchronous validation using interservice communication.

TIME/STEP	DATA STATE	`MSAEmployeeASImp`	`DepartmentDEC`	`MSADepartmentASImp`	`EmployeeDEC`
0	dep. 17 active and has no active employees				
1		REIVE HTTP: register employee 153 at department 17		RECEIVE HTTP: deregister department 17	
2		SEND HTTP: *check department 17 is active* [check OK]		SEND HTTP: *check department 17 is active and has no employees* [check OK]	
3				delete department 17	
4	dep. 17 inactive;	create employee 153 at department 17		SEND ASYNC: *department 17 deregistered*	
5	dep. 17 inactive; emp. 153 active	SEND ASYNC: *new employe 153 at department 17*			
6			RECEIVE ASYNC: *new employee 153 at department 17* check department *x* existence { if (!exists || ! active): SEND ASYNC *deregister employee y*; else do nothing;}		RECEIV ASYNC: *department 17 deregistered* check active employees of department x { if (has active employees) SEND ASYNC *activate department x*; else do nothing; }
7			RECEIVE ASYNC: *activate department 17* check department *x* existence { if (exists && ! active): activate department *x* and SEND ASYNC: department *x* registered); else do nothing;}		RECEIVE ASYNC: *deregister employee 153* Check employee x existence { if (exists && active): deactivate employee x and SEND ASYNC: employee x deregistered); else do nothing;}
8	dep. 17 active; emp. 153 inactive				

that sends the message informing of the employee creation, and the department DEC, which receives the message, validates the department and runs the compensating transaction, are not included, nor are those related to the department deregistration.

Not performing validations that involve interservice communication can be an extreme implementation technique that maximizes the chance of data inconsistencies. Richardson prefers to perform the validations in the context of sagas [30], regardless of their untrustworthiness.

And what happens if the same message is delivered twice due to some error in the message broker? Note that this could be problematic, especially in the implementation of sagas. To avoid this type of problem, microservices business logic must be *idempotent*, i.e., calling several times the same function with the same parameters has no additional effects [30], [36]. In the examples provided in this paper, this ability is delegated in the JMS message broker capacity for detecting duplicate messages [68]. I.e., the business logic does not need to be *idempotent* because the message broker does not send duplicate messages. If this solution is not applied, then application-dependent techniques for achieving idempotency, such as those described in [30] and [36], should be applied.

C. LOCAL FOR SOA AND SAGA FOR MICROSERVICES. TWO TABLES MODIFIED

Let us consider the previously mentioned use case of *client registration* in a bank. The use case gathers the client data, creates the user, and creates a new account with a balance of 0$, because an active user must have an active account. Figure 35 depicts the activity diagram for this use case.

Figure 36 depicts the class diagram for the SOA solution. Figure 37 depicts the sequence diagram for the SOA solution. Note that the client application service is able to create and persist both the client and the account entities. No further

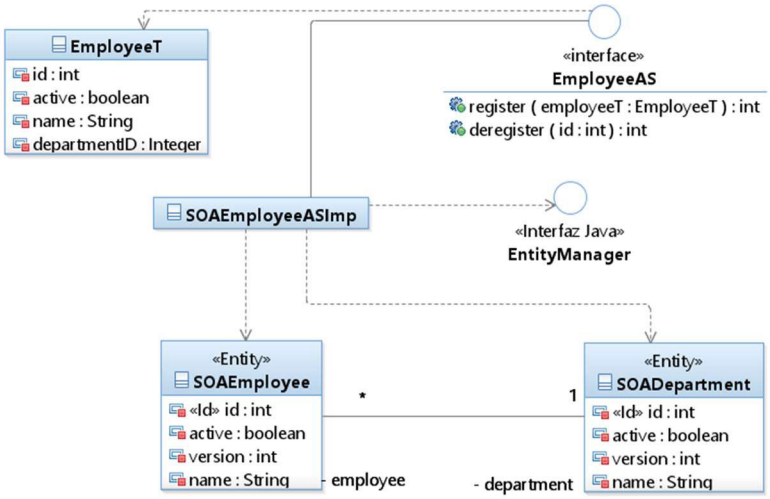


FIGURE 31. Class diagram for the SOA solution of the use case employee registration.

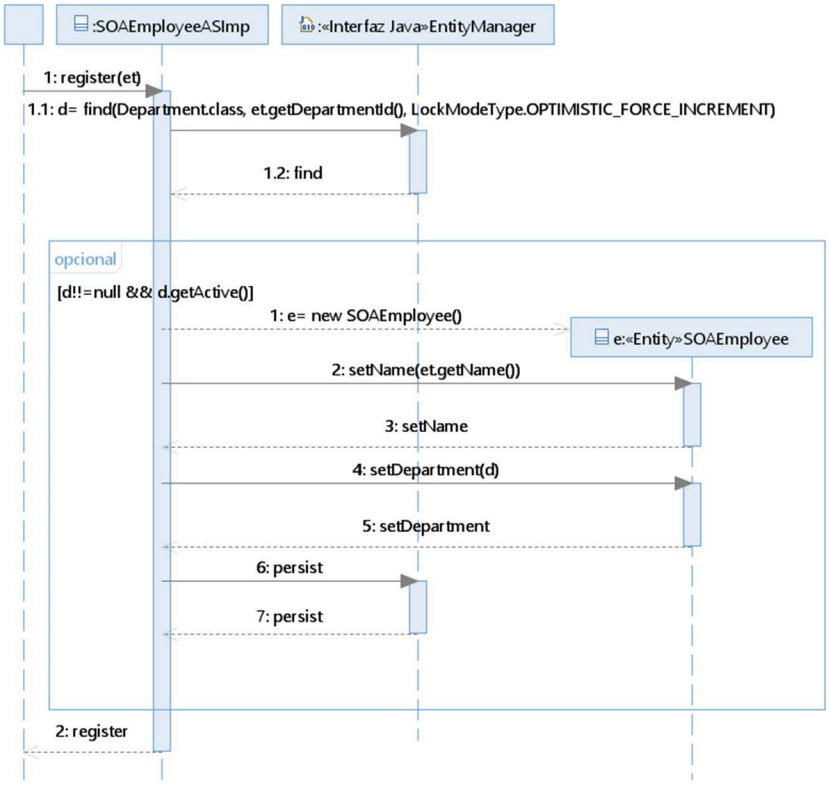


FIGURE 32. Sequence diagram for the SOA solution of the use case of employee registration.

validations or use of JPA owning sides are made and only one client is considered per account for the sake of simplicity. Deactivated clients are not considered either.

In this case, the microservice solution needs a saga because two tables belonging to two different services are modified. Therefore, the saga coordinator should send two asynchronous messages, one to create the client and one to create the account, and then confirm or cancel the client registration depending on the result of each operation. For the sake of brevity, only one example of saga is included in Section V-E.

D. GLOBAL FOR SOA AND SAGA FOR MICROSERVICES. TWO TABLES INVOLVED

Let us consider the use case of *money investment* in a bank. A registered client in a bank wants to invest an amount of money in an investment fund. The use case takes the client identifier, the account identifier, the amount to invest and the

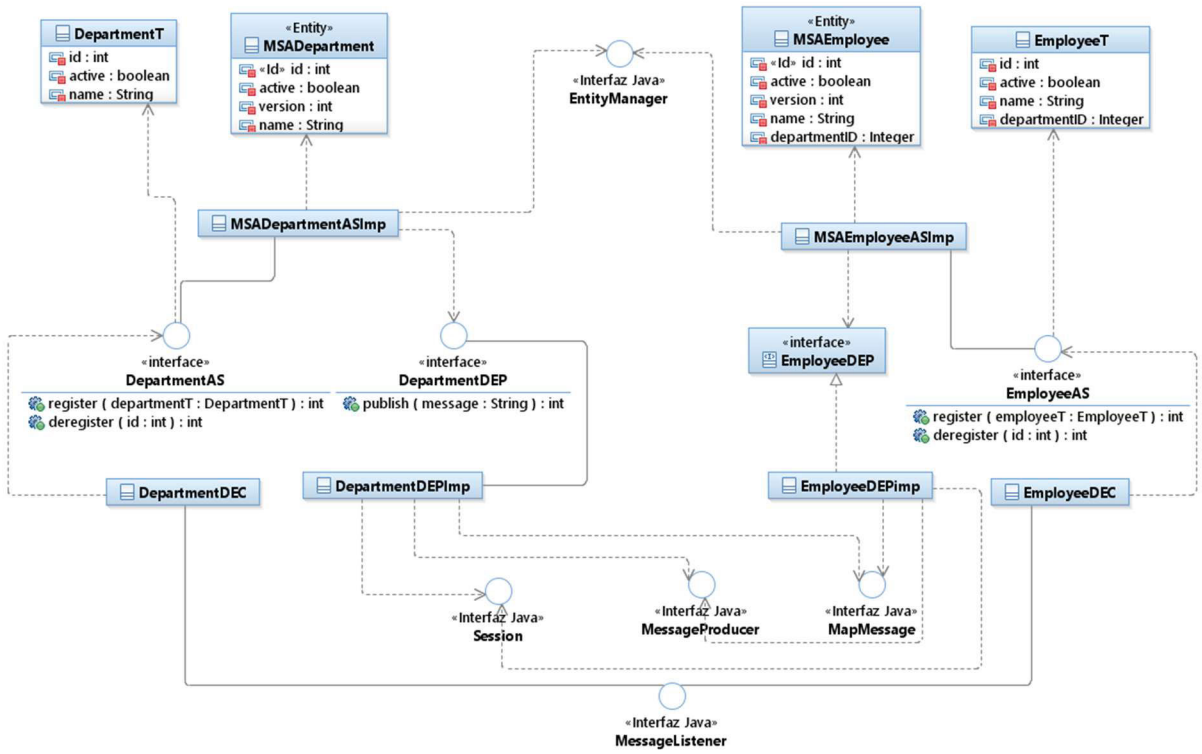


FIGURE 33. Class diagram for the microservices solution of the use case of employee registration.

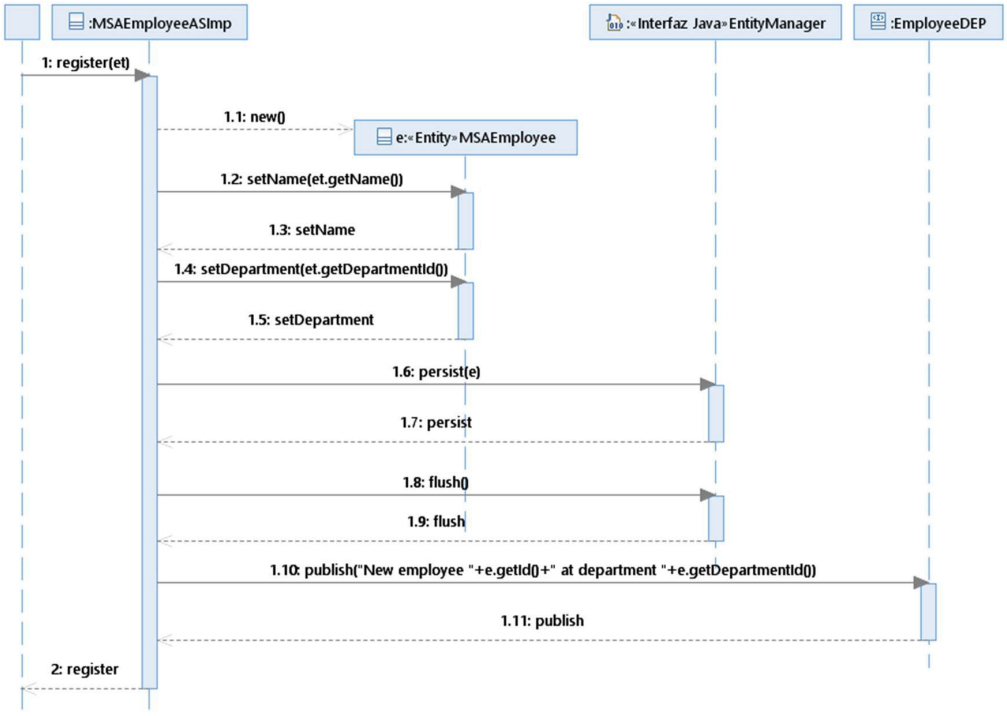


FIGURE 34. Sequence diagram for the microservices solution of the use case of employee registration.

identifier of the investment fund product to be contracted. Then, the application: (i) checks that the client exists and is active; (ii) checks that the account exists, is active, belongs to the client and has enough money to support the investment; (iii) if everything is OK, subtracts the amount from the account's balance; and (iv) contracts the investment fund

TABLE 3. Business logic with no validation using interservice communication.

TIME/STEP	DATA STATE	MSAEmployeeASImp	DepartmentDEC	MSADepartmentASImp	EmployeeDEC
0	dep. 17 active and has no active employees				
1		REIVE HTTP: register employee 153 at department 17		RECEIVE HTTP: deregister department 17	
2		create employee 153 at department 17		delete department 17	
3	dep. 17 inactive; emp. 153 active	SEND ASYNC: *new employe 153 at department 17*		SEND ASYNC: *department 17 deregistered*	
4			RECEIVE ASYNC: *new employee 153 at department 17* check department *x* existence { if (!exists || ! active): SEND ASYNC *deregister employee y*; else do nothing;}		RECEIV ASYNC: *department 17 deregistered* check active employees of department x { if (has active employees) SEND ASYNC *activate department x*; else do nothing; }
5			RECEIVE ASYNC: *activate department 17* check department *x* existence { if (exists && ! active): activate department *x* and SEND ASYNC: department *x* registered); else do nothing;}		RECEIVE ASYNC: *deregister employee 153* Check employee x existence { if (exists && active): deactivate employee x and SEND ASYNC: employee x deregistered); else do nothing;}
6	dep. 17 active; emp. 153 inactive				

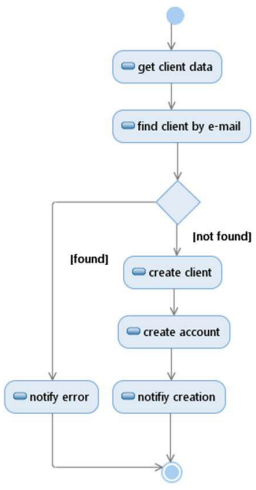


FIGURE 35. Activity diagram for the use case of client registration.

providing the same data that the use case receives. In this case, the process that implements the contract of the investment fund is deployed in a different machine than that of the process that performs the validation and money withdrawn from the account. Figure 38 depicts the activity diagram for this use case.

Figure 39 depicts the class diagram for the SOA solution, and Figure 40 depicts the sequence diagram for this solution. Note that the process that performs the validation invokes the process that contracts the fund as if they were in the same machine. The class that implements the investment interface in the client is responsible for creating and using the JAX-WS proxy that gives access to the remote service. No business delegate has been included for the sake of simplicity. The detailed implementation of this class, as well as the implementation of the remote service that contracts the fund product are not included because they are not illustrative for the transaction processing, as this processing is declaratively defined. Thus, as in the previous examples, the transaction management is hidden because a declarative approach with transaction attributes is supposed. In any case, since a global transaction using a 2PC protocol underlies this transaction, both the account subtraction and the investment fund contract are atomically made, i.e., both are made, or neither of them is made. Note that the optimistic lock can produce failures after agreeing the commit, but these failures can be managed during the exception handling, minimizing the effective time of inconsistencies.

Regarding the microservice solution, again a saga is needed. Therefore, the saga coordinator should send three asynchronous messages for the client validation, the account

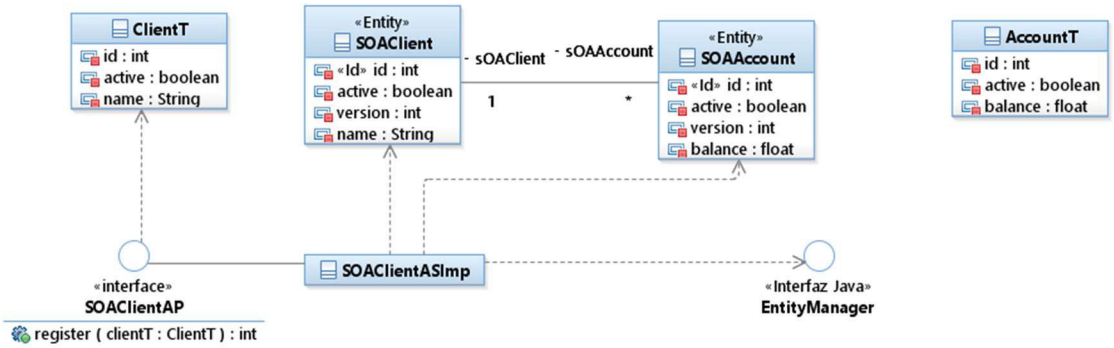


FIGURE 36. Class diagram for the SOA solution of the use case of client registration.

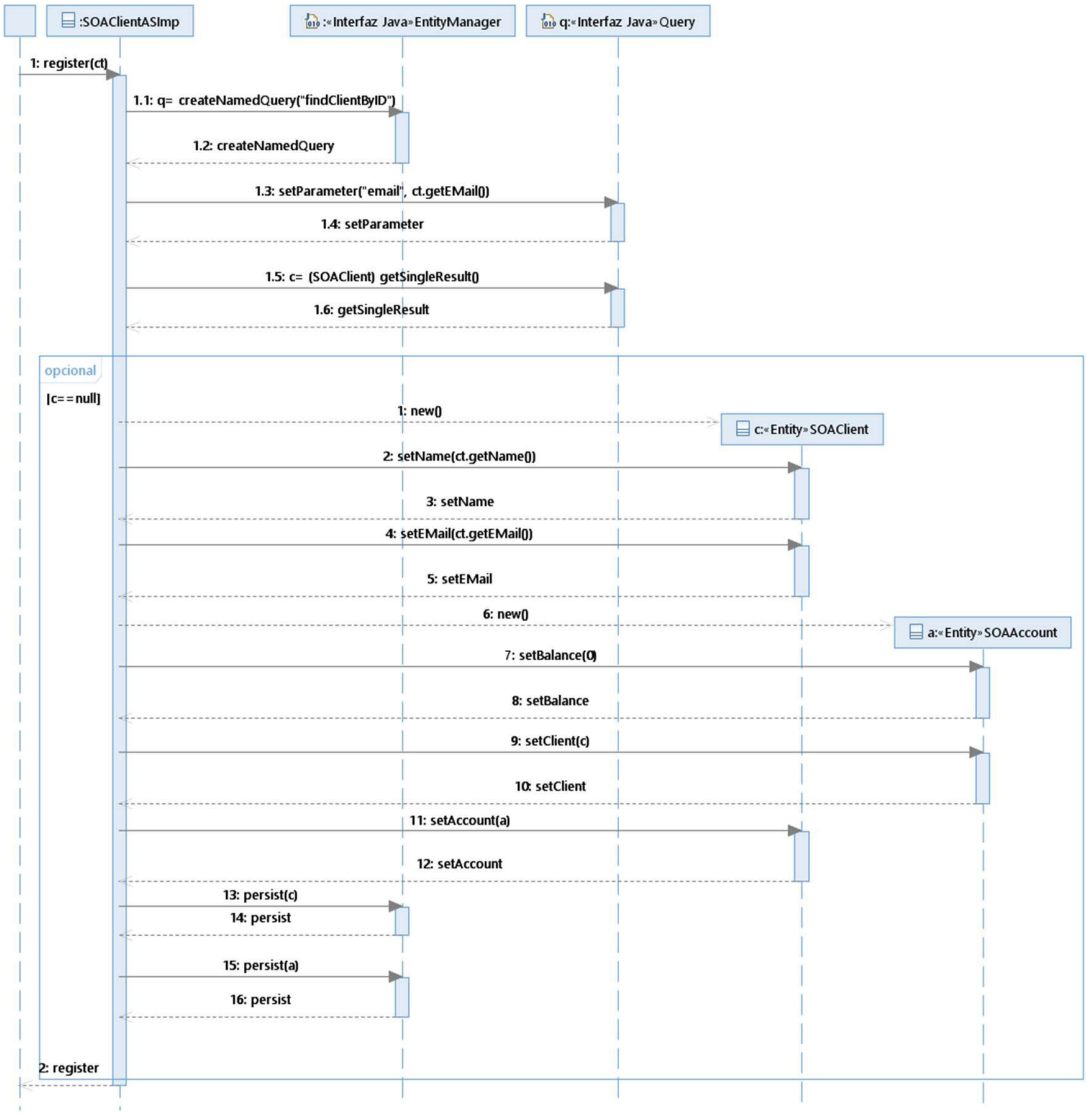


FIGURE 37. Sequence diagram for the SOA solution of the use case of client registration.

validation (and update) and the fund contract, and finally confirm or cancel the client investment depending on the overall result. For the sake of brevity, only an example of saga is included in Section V-E.

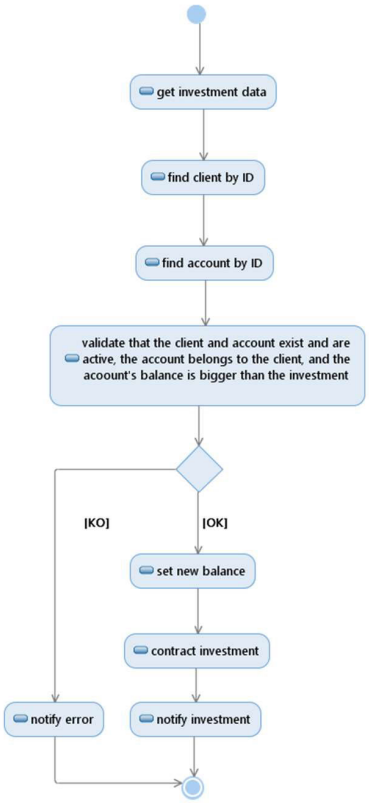


FIGURE 38. Activity diagram for the use case of money investment.

E. LONG-RUNNING. SAGA FOR SOA AND MICROSERVICES. THREE TABLES INVOLVED

This last example considers a long-running transaction that forces the presence of a saga in both SOA and microservices. Let us consider the use case of *travel reservation*. The use case needs the client identifier, the departure and return dates, the origin and the destination. A hotel and a flight are searched using two remote services that are called asynchronously. These services can take the amount of time that they need, up to one day, in order to choose the best offer. Finally, the user can accept or reject the travel reservation, i.e., both the hotel and the flight. Figure 41 depicts the activity diagram for this use case.

This is a long-running transaction that needs a saga to be solved. There are two main ways of implementing sagas: orchestration (a central coordinator manages the long-running transactions and their participants) and choreography (saga participants organize themselves). Although both solutions have advantages and drawbacks, orchestration seems to have more advantages for complex transactions [30], [58]. Therefore, this paper illustrates the use of saga orchestrators called saga coordinators, as in the metamodel depicted in Figure 8.

According to Richardson state machines are the best way to characterize the behavior of sagas [30]. However, Richardson's state machines mix data states and actions in the state machine's states. It is simpler to consider only data states as states in the machine, events launched by the processes as events in the machine, and actions as operations to be performed in the machine. Figure 42 depicts the state machine that characterizes this saga. Note that initially the transaction is started. After that, at any time any event confirming (commit) the hotel reservation or the flight reservation can happen. After that, if the complementary process commits, the saga just waits for the user's confirmation. When this happens, the saga ends successfully. Other combinations of events lead to the saga cancellation. The user's confirmation or cancellation only takes effect when both the hotel and the flight have been reserved.

Although every saga is different, if each operation performed in a saga is numbered, then the states can reflect the number of operations committed. Figure 43 reflects an equivalent state machine to those depicted in Figure 42 using this approach.

Figure 44 depicts the class diagram for implementing the saga in both SOA and MSA.

The travel reservation application service redirects the HTTP API request for travel reservation to the travel reservation saga coordinator. This coordinator is responsible for implementing the state machine in Figure 42. It uses hotel and flight saga command publishers for sending asynchronous messages to the hotel and flight services using JMS as well as a travel domain event publisher for sending domain events using JMS. It also uses a travel reservation saga coordinator reply handler for managing the asynchronous responses (commit or rollback) of the hotel and flight services.

Hotel and flight application services use saga command handlers for processing the asynchronous request sent by the saga coordinator. When these processes finish their tasks, they use saga reply publishers to send the asynchronous commit/rollback to the saga coordinator reply handler, which redirects the asynchronous request to the travel reservation saga coordinator.

Because only one class is going to be responsible for managing the asynchronous messages sent from the invoked application services to the saga coordinator reply handler, in this case JMS queues are used for asynchronous communication.

The core of this solution is the travel reservation saga coordinator, and the sequence diagram focusses on it. Figure 45 depicts the sequence diagram for the travel reservation. In this case, the saga coordinator sends both messages without waiting for a response, but other saga coordinators could send only one message and use a state machine for sending new messages taking into account the response to the initial message. Figure 45 starts a parallel execution (due to the asynchronous execution of the messages sent by the processes that receive the hotel and flight reservations), but this is not mandatory for saga coordinators, which can enforce the sequential execution of several microservices according to the state machine that controls the saga. Figure 46 depicts the sequence diagram for the hotel commit. Figure 47 depicts the sequence diagram for the flight rollback. Figure 48

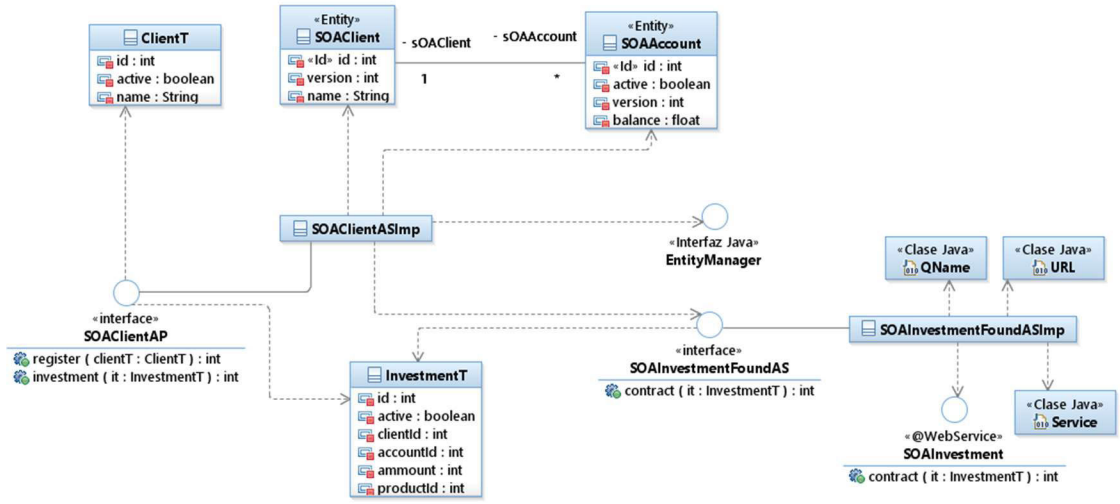


FIGURE 39. Class diagram for the SOA solution of the use case money investment.

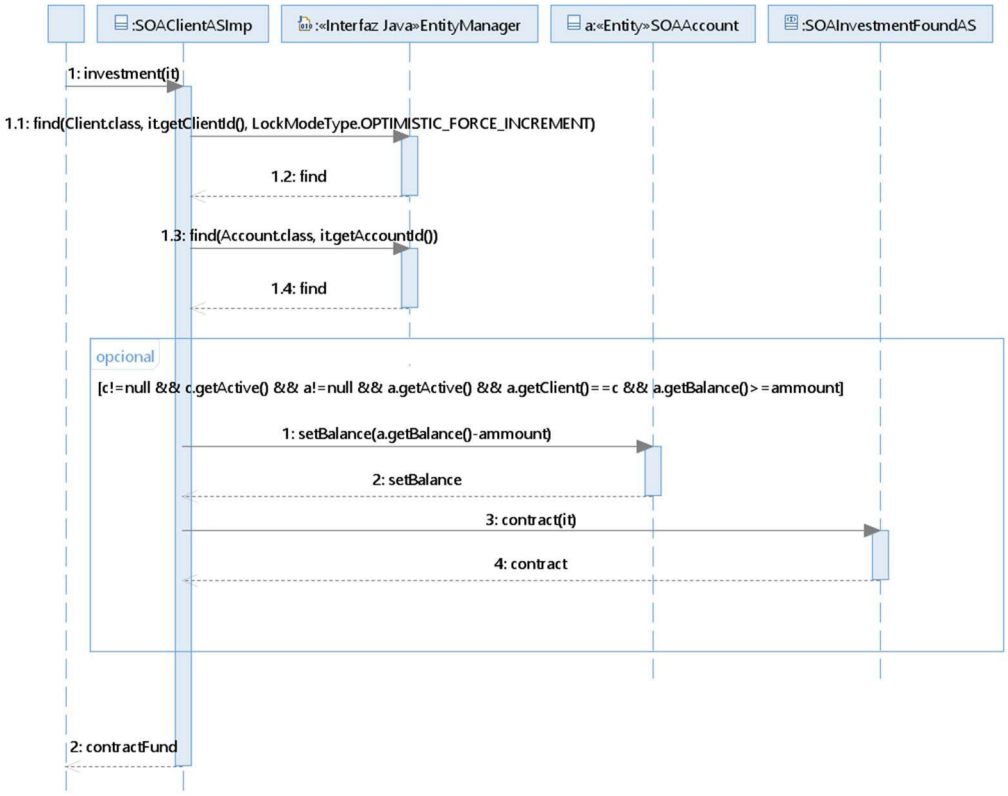


FIGURE 40. Sequence diagram for the SOA solution of the use case of money investment.

depicts the sequence diagram for the user confirmation. Note that the entity/table that stores the travel reservation takes care of the state of the saga transaction, and every event/operation actualizes this state. A specific transaction class could also be used to manage the saga. In Figures 46-48 an additional case could be included for events other than those considered: doing nothing or launching an exception.

As in the previous examples, declarative transaction management is supposed in every transactional operation of every class and is not made explicit in the sequence diagram.

The travel reservation saga coordinator inbound adapter could receive two asynchronous commit/rollback messages simultaneously. This issue is not related to idempotency, but to simple concurrent programming. The travel reservation

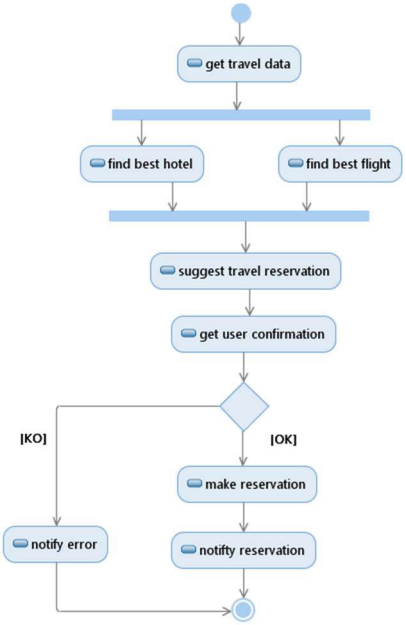


FIGURE 41. Activity diagram for use case of travel registration.

entity in Figure 44 has a version attribute that takes care of its simultaneous modification. In the case of optimistic lock exception, the travel reservation saga coordinator will keep retrying the conflicting operation until it is processed correctly.

One problem with sagas is the lack of isolation between the operations that different sagas can perform against the same data, such as running concurrently travel reservation and travel cancellation use cases. Richardson proposes six countermeasures for this lack of isolation [30]:

- *Semantic lock*: an application-level lock.
- *Commutative updates*: design update operations to be executable in any order.
- *Pessimistic view*: reorder the steps of a saga to minimize business risk.
- *Reread value*: prevent dirty writes by rereading data to verify that it's unchanged before overwriting it.
- *Version file*: record the updates to a record so that they can be reordered.
- *By value*: use each request's business risk to dynamically select the concurrency mechanism.

In the saga included in this paper, semantic lock is used (e.g., a travel cancellation cannot be performed until the travel reservation entity has the confirmed state), with no retry and with reread value implemented using JPA version control.

This is a particular implementation strategy. Other strategies could have used a specific table to manage the saga state and store every event there. In addition, the saga coordinators and their incoming/outgoing adapters could use a more abstract design with reuse in mind, as some frameworks that implement saga coordinators do. In this example, this paper chooses the implementation that seems to be more explicit for understanding the core elements that participate in sagas implementations, omitting even the use of the *state* pattern [53]. In this line, this paper does not use a stack of events that stores all the saga-related events, unlike other authors [30], [36]. Finally, no use has been made of WS-Business Activity facilities in order to provide an example reusable for both SOA services and microservices.

VI. DISCUSSION

A. CURRICULA RECOMMENDATIONS

This paper illustrates the high complexity of *service transaction management. To fully understand what happens in a *service application at a transactional level, and focusing only on the JEE platform, it is necessary to know multitier and microservice architectures, JPA, JAX-WS, JAX-RS, JMS, JEE security and local, global and saga transactions.

How are these concepts considered in present curricula recommendations? CS2023 provides more details about transaction management than *Computer Science 2013* [69] but focused only on *Data Management* and *Parallel and Distributed Computing* knowledge areas, as well as moving contents from the *Software Engineering* knowledge area to *Data Management*. *Software Engineering 2014* [70] focuses entirely on software engineering and pays almost no attention to transactional management [1]. *Information Systems 2010* [71] had a core course on enterprise architecture, but *Information Systems 2020* [72] removed it from its competency areas [1].

*services and their transaction management are at the core of enterprise software development and present ACM/IEEE curricular recommendations do not pay special attention to this issue. Maybe ACM/IEEE consider this topic very specific for graduate curricula, but *services and their transaction management are very important issues for the development of key applications such as banking, commercial or healthcare systems.

My previous paper [1] suggested the creation of a course focused on service transactions that could be named *Enterprise Software Architecture*. The main core-1 outcome of that course was "to build a n-tier application using service transactions". A core-2 outcome was "to build an n-tier application using global service transactions with two-phase commit". The material provided in this paper could be the basis for another course on *Advanced Enterprise Software Architectures* dedicated to explaining *services and their transactions to students. The course should also include secure service communications analyzed in Section VI-D. This course would have two main outcomes: (i) to build a SOA application that involves global transactions and sagas; and (ii) to build an MSA application that involves sagas. JPA should be used in both cases.

They are two courses, which translates into 6 US credits (12 ECTS credits), or equivalently, 120 course hours. Such coverage for these courses is not found in ACM/IEEE curricula recommendations, although they deal with other key issues in software development nowadays.

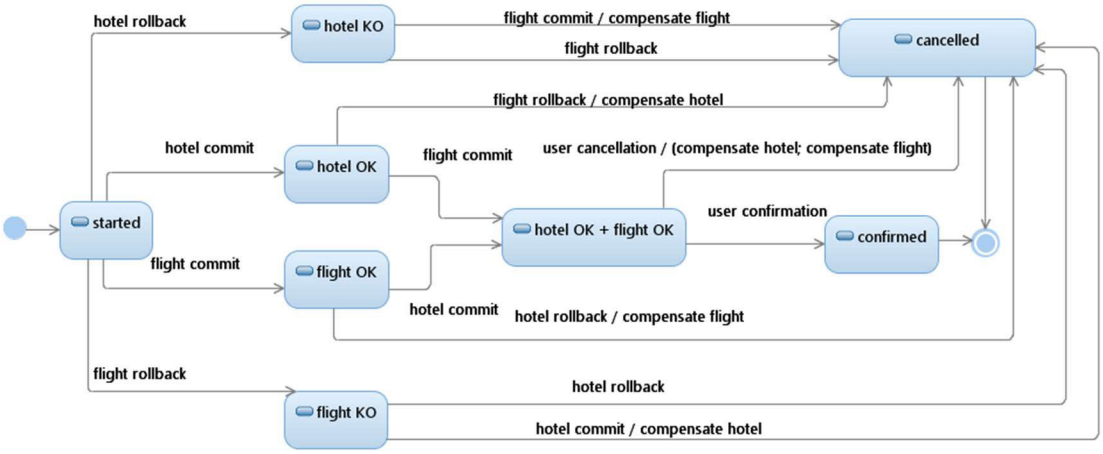


FIGURE 42. State machine for the saga that coordinates the solution of the use case of travel reservation.

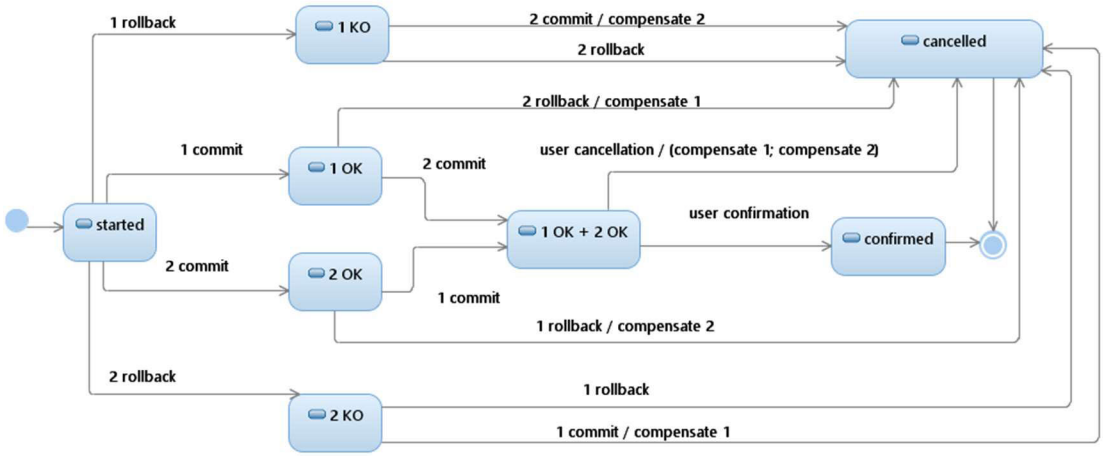


FIGURE 43. State machine of Figure 42 where operations have been replaced by numbers (1-hotel; 2-flight).

B. SOA TRANSACTION MANAGEMENT VS. MICROSERVICES TRANSACTION MANAGEMENT

Regarding *service programming and its transaction management, SOA services need complex transaction managers, as the one defined in this paper, but they allow to lighten the code written in the application for transaction management. Thus, WS-* standards and their implementation in SOA application servers and frameworks make life easier for application developers. On the contrary, microservices need simple transaction managers, but their transaction management is much more complex than SOA services, needing more code for transaction management in the application.

The existence of a common database for the SOA services and the global transactions relegates the sagas to long-running transactions only, as the examples provided in this paper have depicted. On the contrary, the pattern single database per service forces that every single operation against data that involves modifications in more than one table becomes a saga in microservices. And sagas are complex artifacts. Of course, automated frameworks can be used to simplify the saga implementation, but the state machine that controls the saga must be designed for every saga, and these state machines can be extremely complex when a significant number of services are invoked from the saga coordinators. In addition, the application becomes tied to the additional framework that supports sagas.

As Zhou et al. state, ‘‘MSA should not be adopted blindly due to ‘everyone is doing it’ and it must be comprehensively understood before deciding to migrate to MSA’’ [65]. However, it should be mentioned that SOA global transactions rely on a few frameworks capable of interacting with complex transaction managers to generate SOAP messages with WS-Atomic information, while microservices rely on simpler, widely available underlying technologies.

The true problem with MSA is the database per service. Amazon Web Services, AWS, considers the *shared database per service* pattern [60], which softens the problems bound to the database per service. But if sagas and events are no longer

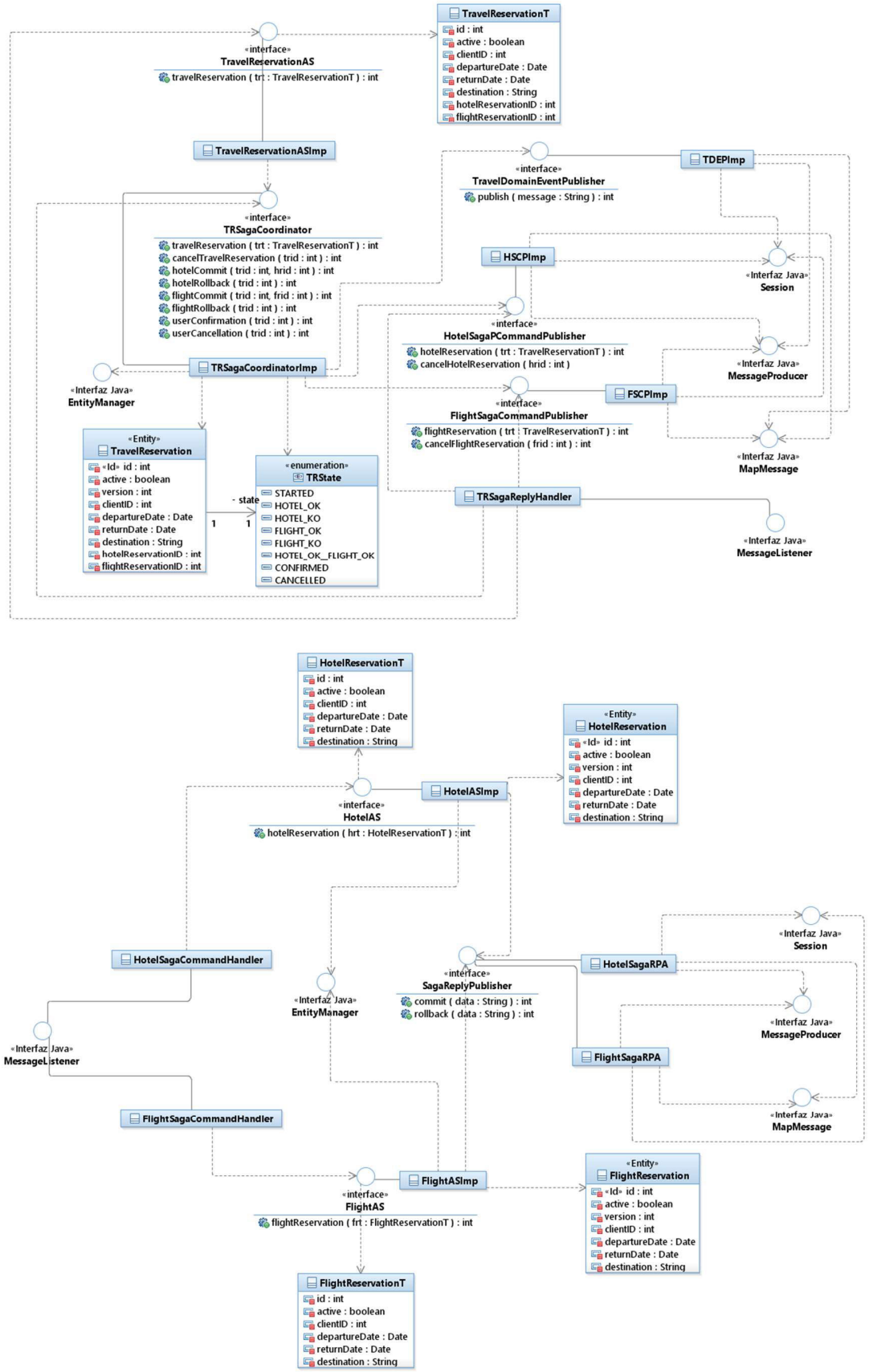


FIGURE 44. Class diagram for the solution to the use case of travel reservation.

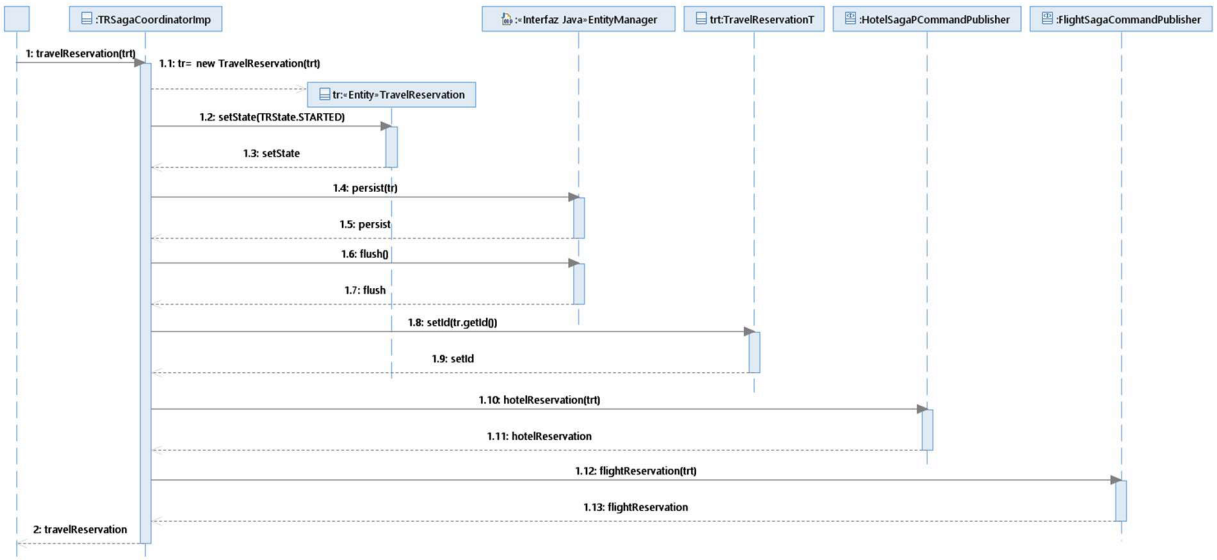


FIGURE 45. The saga's coordinator performs the travel reservation of the use case of travel reservation. Note that in this use case, the business logic is split into this operation and the responses of the invoked processes according to Figures 46, 47 and 48.

needed, microservices mutate into something that can be defined as *fine-grained service architecture*, which coincides with the column SOA in Table 1, but using the application service as a deployment unit instead of the whole application. This solution avoids the use of sagas (except for long-running transactions), but it forces all the services to depend on the same database schema, with the availability issues this can cause.

Note also that, as the examples in this paper have shown, unlike SOA services, microservices need a specific type of business logic implementation to the extent of questioning business logic validations. This, along with the lack of atomicity and isolation of sagas, means that business logic can generate data inconsistencies that have to be solved later.

The main adversary for the 2PC used in SOA is the *CAP Theorem* [73], which states that any distributed data store can only provide two of the following guarantees: *consistency*, *availability*, and *partition tolerance*. If high availability is preferred to consistency, then MSA is the right architecture. The high availability of microservices relies on asynchronous communication among them and single databases per service, which allows some services to continue running while others are down. However, in a transactional environment, sagas usually entail a synchronous execution of services which communicate asynchronously. For example, considering the employee registering use case of Section V-B, if a saga is used to check the department before the employee registration, although the saga coordinator sends an asynchronous message to the microservice that validates the department, the saga's state machine keeps waiting for an asynchronous response from the microservice that validates the department, and does not register the employee until this response is received. A saga-less solution, as the one depicted in Section V-B, can be provided, at the cost of increasing the risk of data inconsistencies: the REST API can provide a synchronous answer to the user informing about the employee registration, and later, after domain events are processed, the user is informed that the employee has not been finally registered. Note that a saga minimizes this risk of inconsistency, but does not prevent it, as the validation of the department is not atomically included as a part of the same transaction as the employee registration, unlike in the SOA solution.

Thus, although better maintainability, testability, deployability, scalability and fault isolation are among the advantages claimed by proponents of MSA, these theoretical advantages must be balanced against the higher number of functions, inter-service communications, events, compensating transactions, sagas and database connections that arise in the MSA-based implementation of EAs compared to SOA. Functions, events, compensating transactions and sagas must be programmed in the present and maintained in the future.

As Sommerville states, microservices are not encouraged for those applications where consistency is paramount [58], although, in practice, web services are offered for banking applications [74].

C. ALTERNATIVE TRANSACTION MANAGEMENT MODELS AND HYBRID ARCHITECTURES

This paper considers only two transaction management models: 2PC and eventual consistency. 2PC is the classic model underlying application servers [3], [4], [5] and is used in multitier architecture [50], [51]. Eventual consistency management is the alternative model used by microservices and is implemented with sagas [30], [62], [63], [64].

IBM, Oracle, RedHat and Microsoft are among the biggest players in the application server market [75]. These companies are not only major providers of application servers,

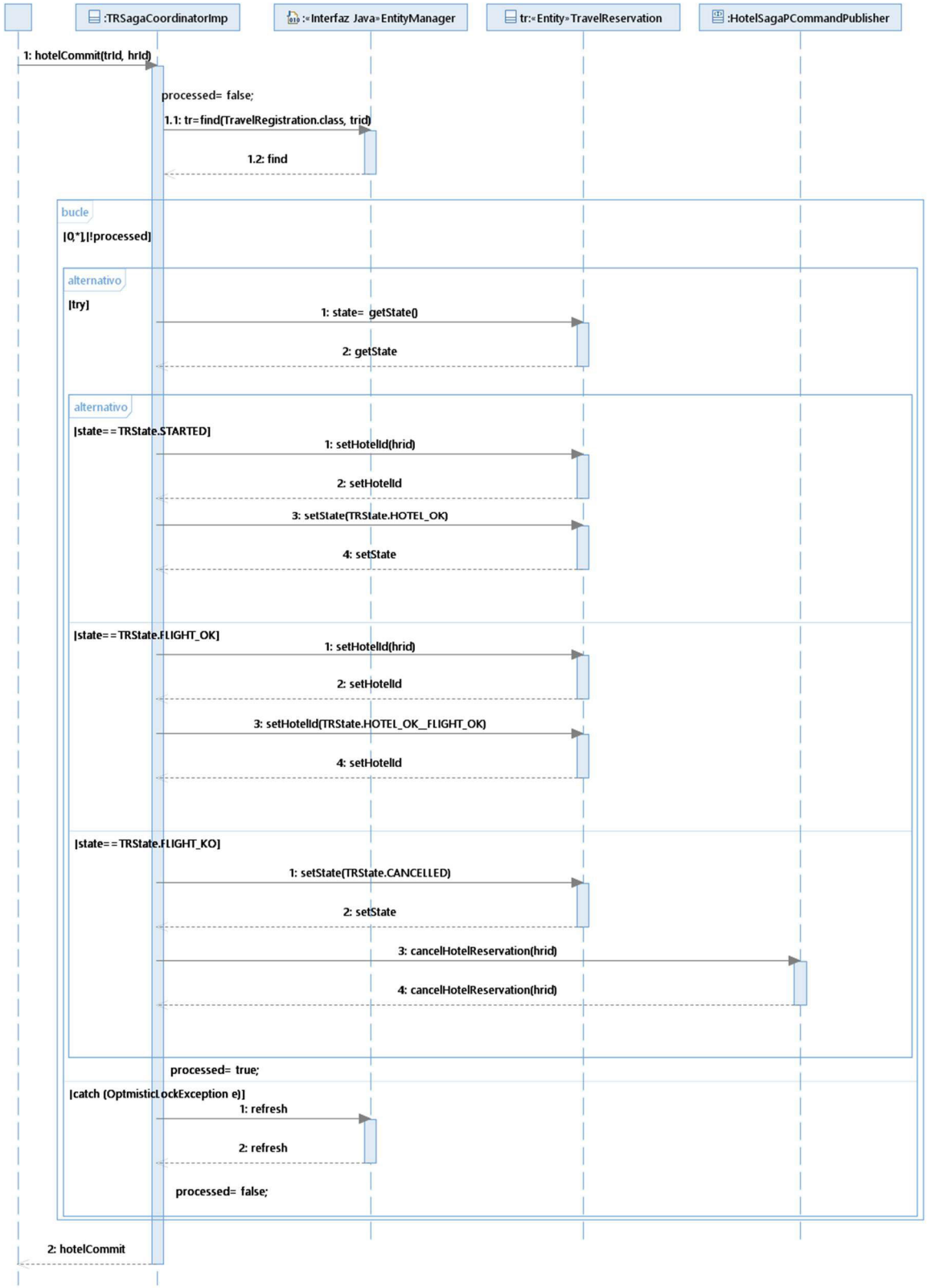


FIGURE 46. The saga's coordinator receives a hotel commit and behaves according to the state machine of Figure 42.

but they also rank among the largest suppliers of cloud computing infrastructure, alongside Amazon Web Services (AWS). Therefore, large entities, which rely on these players for the development and deployment of their EAs,

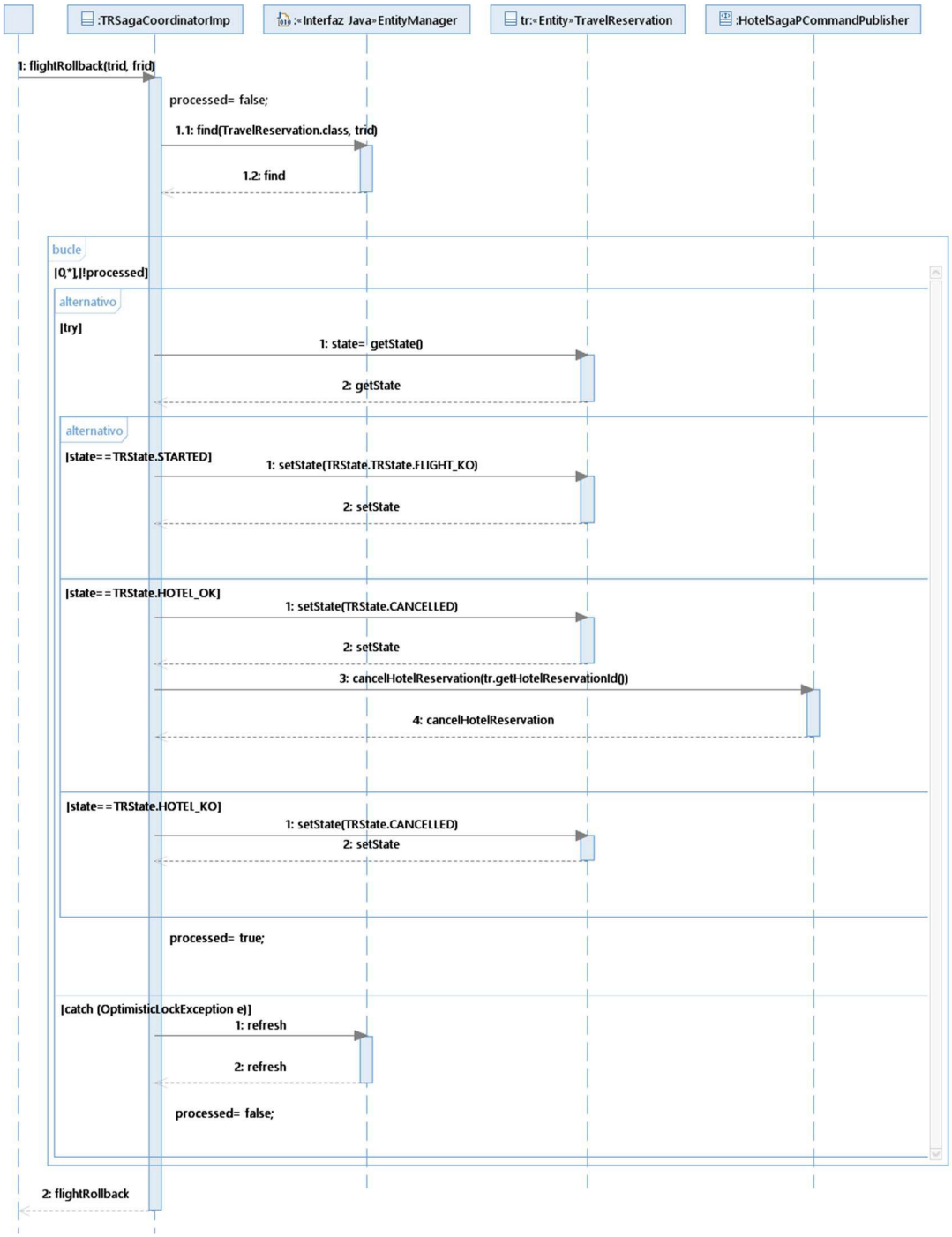


FIGURE 47. The saga's coordinator receives a flight rollback and behaves according to the state machine of Figure 42.

can choose between SOA and application server deployment, or MSA and container-based/serverless deployment, since these are the main options offered by vendors/implementors.

There are other approaches for distributed transactions such as *TCC*(Try, Confirm, Cancel), *local messaging*, *transactional messaging*, *best-effort notification*, and *AT* [76], but these approaches are not considered in the SOA or MSA pattern catalogues on which major players in enterprise application development base their architectures [50], [51], [62], [63], [64], although Richardson does consider transactional messaging [30]. Therefore, although the transaction management models have been characterized in this paper using theoretical models, in practice, most of the enterprise

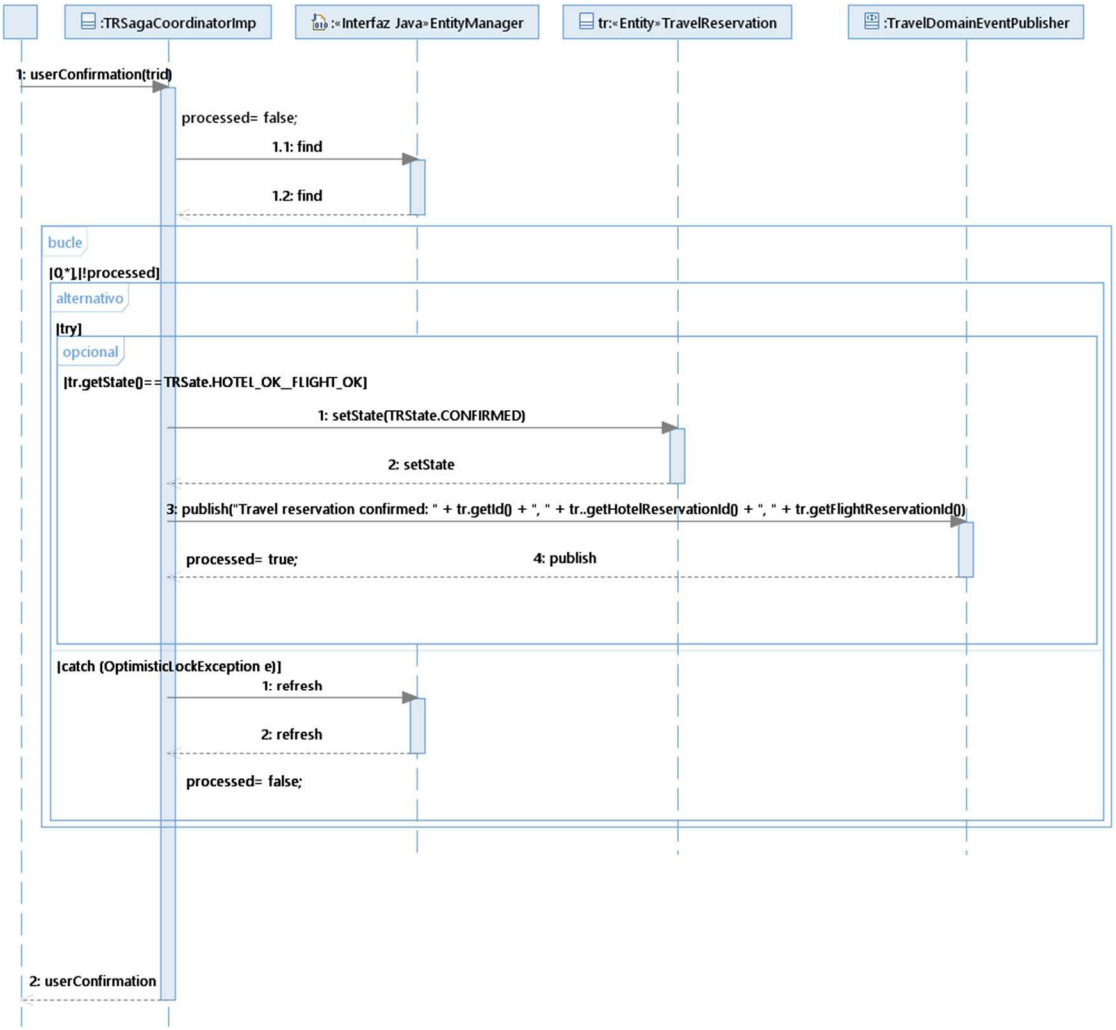


FIGURE 48. The saga's coordinator receives the user confirmation and behaves according to the state machine of Figure 42.

applications are developed using SOA services with 2PC protocol or using microservices with sagas.

Today, however, a new type of software architecture is emerging in which SOA services and microservices coexist. This type of architecture is called *hybrid architecture* and has benefits and drawbacks. The benefits are incremental migration, leveraging existing investments, improved scalability and flexibility and innovation [77]. The drawbacks are: complexity, data consistency and operational overhead [77].

Data consistency is directly related to transaction management. It is evident that the problem arises when SOA services are interleaved with microservices in the same transaction. In this case, there is no other solution than using eventual consistency management and sagas, because microservices do not participate in 2PC transactions. To this end, SOA services should develop the software infrastructure depicted in Figure 44 to be able to participate in sagas.

Regarding cloud hybrid architectures, this is a system-level architecture that is beyond the scope of this paper.

D. SERVICE SECURITY

The examples provided in Section V do not include any detail about security, which is a critical aspect of enterprise applications that handle sensitive information. Reference [66] analyzes the most significant software security patterns for enterprise applications. This section considers those directly involved in *services security. Note that, because security is an orthogonal concept to functionality, most development frameworks and applications services provide mechanisms for implementing security patterns without affecting the application code, such as the one depicted in Section V.

Secure pipe guarantees the integrity and privacy of data sent over the wire [78]. In the context of HTTP connections, this solution is implemented as *Secure HTTP* (HTTPS) [78]. Therefore, all the connections performed in the examples depicted in Section V should be performed on HTTPS connections.

Point-to-point secure invocation of web services requires several patterns [66]. The most important are *secure session*

object [78], that abstracts the encapsulation of authentication and authorization credentials that can be passed across boundaries, and *container-managed security* [78], that enforces the use of credentials for invoking services. Therefore, when clients invoke services in the examples of Section V, those invocations must include secure session objects that allow containers to grant access to the invoked services.

End-to-end secure invocation tries to overcome the limitations of point-to-point secure invocation by using certificates at both sides of the services to encrypt outgoing and incoming messages [66]. This is a very restrictive solution that should be carefully analyzed before being put into practice.

Code injection attacks [66] to the services should be avoided including *message interceptor gateways* [78] and *message inspectors* [78].

Logging the service layer is often necessary [66], and *secure service proxies* [78] can be used for this purpose.

VII. CONCLUSION

*service transactions involve a significant number of complex concepts that may require specific subjects in curricula recommendations. Present curricula recommendations do not cover the topics needed to manage *service transactions. This paper provides a strong guidance for those lecturers who want to teach enterprise application architectures to their undergraduate or graduate students. The paper defines MOF metamodels for multitier and hexagonal architectures, plus a mapping between them. The paper also defines MOF metamodels for SOA/multitier services and microservices, and defines the concept of *services for encompassing both types of remote services, as well as a MOF metamodel for *service transactions, and a transaction manager able to perform global transactions among SOA services. MSA services can reuse this transaction manager or the simpler one defined in my previous paper. In addition, this paper also includes five examples of *service transactions including local, global and saga transactions, which compare the differences between SOA and MSA at the transactional level.

Regarding transaction management, and considering the advantages and drawbacks discussed in the previous section, MSA might not be the right choice for EAs. It can work, and it does, but it is not ideal. It has taken too much effort to make efficient transactional management in EAs reach such a level of refinement and optimization that a declarative management of transactions is performed. And now a regression is taking place, where a complex network of events, sagas, compensating transactions, inter-service communication, and multiple database connections are needed for dealing with simple functions against databases, such as employee creation.

Netflix is commonly proposed as the paradigmatic example of success using MSA. But Netflix's business has to deal with lots of heavy user connections for supporting streaming, and it is not a prototypical case of EA.

Amazon turned its EA into a microservice-based application. But it took two years of development and Amazon is a leading provider of microservices infrastructure. Asking Amazon whether MSA is the right architecture for its application is like asking Microsoft whether Windows is the right operating system for the PCs they use. And even Amazon recognizes the possibility of having shared databases, which eliminates the DBPS, the real problem of microservices in EAs.

Microservices have benefits, but their deployment advantages do not outweigh the problems they pose in a genuinely transactional EA. Thus, preferring SOA architecture for EA development over microservices is as reasonable as preferring microservices for other types of domains, such as distributed systems with significant hardware use (e.g., Netflix).

The fine-grained service architecture defined in this paper is a compromise between SOA and microservices, allowing the independent deployment of services while maintaining a shared database between them, with all the advantages and disadvantages that this entails.

Finally, due to the reasons presented in this paper, migrating from SOA services-based applications to microservices applications becomes very complex because all the transactional business logic has to be rewritten.

Developers need to consider all these issues when deciding which type of *service to choose when implementing enterprise applications. This is not a simple issue, and this paper can help them make a decision.

REFERENCES

[1] A. Navarro, "Fundamentals of transaction management in enterprise application architectures," *IEEE Access*, vol. 10, pp. 124305–124332, 2022, doi: 10.1109/ACCESS.2022.3224759.
[2] ACM/IEEE. (Jan. 2024). *Computer Science Curricula 2023*. [Online]. Available: https://dl.acm.org/doi/pdf/10.1145/3664191
[3] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*. San Mateo, CA, USA: Morgan Kaufmann, 2009.
[4] M. Little, J. Maron, and G. Pavlik, *Java Transaction Processing Design and Implementation*. Upper Saddle River, NJ, USA: Prentice-Hall, 2004.
[5] M. Richards, *Java Transaction Design Strategies*. Toronto, ON, Canada: C4Media Inc., 2006.
[6] OASIS. (2009). *Web Services Atomic Transaction (WS-Atomic Transaction)*. [Online]. Available: http://docs.oasis-open.org/ws-tx/wsat/2006/06
[7] OASIS. (2009). *Web Services Business Activity (WS-Business Activity)*. [Online]. Available: http://docs.oasis-open.org/ws-tx/wsba/2006/06
[8] OASIS. (2009). *Web Services Coordination (WS-Coordination)*. [Online]. Available: http://docs.oasis-open.org/ws-tx/wscoor/2006/06
[9] OASIS. (2009). *Web Services Transaction (WS-TX) TC*. [Online]. Available: https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=ws-tx#technical
[10] *Hypermedia/Time-based Structuring Language (HyTime)*, Standard ISO/IEC 10744:1997, 1997.
[11] A. Dinn, J. Halliday, K. Connor, and M. Little. *Transactions Development Guide*. Redhat. Accessed: Apr. 1, 2025. [Online]. Available: https://access.redhat.com/documentation/es-es/jboss_enterprise_application_platform/5/html/transactions_development_guide/index
[12] B. Kratz. (2004). *Protocols For Long Running Business Transactions*. nfolab Tech. Rep. Ser. [Online]. Available: https://www.researchgate.net/publication/239546418_Protocols_For_Long_Running_Business_Transactions
[13] M. Yuan, Z. Huang, and F. Xiao, "Modeling and analysis of flexible transaction for web services," in *Proc. IEEE Int. Conf. Web Services*, Jul. 2009, pp. 1008–1009.

[14] Y. Xiao and H. Zhang, "SHTM: A semantic hierarchy transaction model for web services transactions," in *Proc. IEEE Asia–Pacific Services Comput. Conf.*, Dec. 2008, pp. 519–523.

[15] I. Saleh, G. Kulczycki, and M. B. Blake, "Formal specification and verification of transactional service composition," in *Proc. IEEE World Congr. Services*, Jul. 2011, pp. 474–481.

[16] R. Casado, J. Tuya, and M. Younas, "An abstract transaction model for testing the web services transactions," in *Proc. IEEE Int. Conf. Web Services*, Jul. 2011, pp. 730–731.

[17] B. A. Schmit and S. Dustdar, "Systematic design of web service transactions," in *Proc. TES*, in Lecture Notes in Computer Science, vol. 3811, 2006, pp. 23–33.

[18] I. Vasquez, J. A. Miller, K. Verma, and A. Sheth, "OpenWS-transaction: Enabling reliable web service transactions," in *Proc. ICSOC*, in Lecture Notes in Computer Science, Jan. 2005, pp. 490–494.

[19] M. Papazoglou and B. Kratz, "A business-aware web services transaction model," in *Proc. ICSOC*, in Lecture Notes in Computer Science, Jan. 2006, pp. 352–364.

[20] P. Dolog, M. Schäfer, and W. Nejdl, "Design and management of web service transactions with forward recovery," in *Advanced Web Services*. Cham, Switzerland: Springer, 2014.

[21] S. Choi, H. Jang, H. Kim, J.-S. Kim, S. M. Kim, J. Song, and Y.-J. Lee, "Maintaining consistency under isolation relaxation of web services transactions," in *Proc. WISE*, in Lecture Notes in Computer Science, Jan. 2005, pp. 245–257.

[22] R. Casado, J. Tuya, and M. Younas, "A family of test criteria for web services transactions," *Proc. Comput. Sci.*, vol. 10, pp. 880–887, Jan. 2012.

[23] S. Choi, H. Kim, H. Jang, J. Kim, S. M. Kim, J. Song, and Y.-J. Lee, "A framework for ensuring consistency of web services transactions," *Inf. Softw. Technol.*, vol. 50, nos. 7–8, pp. 684–696, Jun. 2008.

[24] J. Park and K.-S. Choi, "An adaptive coordination framework for fast atomic multi-business transactions using web services," *Decis. Support Syst.*, vol. 42, no. 3, pp. 1959–1973, Dec. 2006.

[25] G. Pardon and C. Pautasso, "Towards distributed atomic transactions over RESTful services," in *REST: From Research to Practice*. Cham, Switzerland: Springer, 2011, pp. 507–524.

[26] (2013). *REST-Atomic Transactions 2.0. Draft 8, Mark Little*. [Online]. Available: https://www.narayana.io/docs/specs/restat-v2-draft-8-2013-jul-29.pdf

[27] Z. Wei and J. Chen, "Web services asynchronous transaction model based on JMS," in *Proc. Int. Conf. MultiMedia Inf. Technol.*, Dec. 2008, pp. 126–129.

[28] (2022). *Compensating Transaction Pattern. MS Azure Cloud Design Patterns*. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction

[29] S. Petunin. (2021). *A Guide to Transactions Across Microservices*. Baeldung. [Online]. Available: https://www.baeldung.com/transactions-across-microservices

[30] C. Richardson, *Microservices Patterns*. Shelter Island, NY, USA: Manning Publication Co, 2019.

[31] R. Ganji. (2021). *How to Get Closer to Consistency in Microservice Architecture*. DZone. [Online]. Available: https://dzone.com/articles/transaction-management-in-microservice-architectur

[32] K. Xiang. (2018). *Patterns for Distributed Transactions Within a Microservices Architecture*. [Online]. Available: https://developers.redhat.com/blog/2018/10/01/patterns-for-distributed-transactions-within-a-microservices-architecture#

[33] H. García-Molina and K. Salem, "Sagas," in *Proc. SIGMOD*, Jan. 1987, pp. 249–259.

[34] A. Fachat, J. Laredo, and H. Verhoeven. (2022). *Extend Saga to the Cloud for Distributed Transactions*. IBM. [Online]. Available: https://www.ibm.com/cloud/architecture/architecture/practices/extend-saga-to-the-cloud/

[35] O. Başkök. (2019). *SAGA Pattern Briefly*. [Online]. Available: https://medium.com/trendyol-tech/saga-pattern-briefly-5b6cf22dfabc

[36] B. Christudas, *Practical Microservices Architectural Patterns*. New York, NY, USA: Apress, 2019.

[37] Baeldung. (2023). *Saga Pattern in Microservices*. [Online]. Available: https://www.baeldung.com/cs/saga-pattern-microservices

[38] *Axon Reference Guide 2024*. Accessed: Apr. 1, 2025. [Online]. Available: https://docs.axoniq.io/reference-guide

[39] (2023). *Eclipse MicroProfile LRA*. [Online]. Available: https://download.eclipse.org/microprofile/microprofile-lra-2.0/microprofile-lra-spec-2.0.html

[40] (2024). *SEATA*. [Online]. Available: https://seata.apache.org/

[41] (2024). *Introduction to Camunda Platform 8*. [Online]. Available: https://docs.camunda.io/docs/guides/

[42] (2024). *Apache Camel Documentation*. [Online]. Available: https://camel.apache.org/docs/

[43] K. M. Koyya and B. Muthukumar, "A survey of saga frameworks for distributed transactions in event-driven microservices," in *Proc. 3rd Int. Conf. Smart Technol. Comput., Electr. Electron. (ICSTCEE)*, Dec. 2022, pp. 1–6.

[44] B. Ibryam. (2021). *Distributed Transaction Patterns for Microservices Compared*. Red Hat Developer. [Online]. Available: https://developers.redhat.com/articles/2021/09/21/distributed-transaction-patterns-microservices-compared#

[45] M. T. González-Aparicio, M. Younas, J. Tuya, and R. Casado, "A transaction platform for microservices-based big data systems," *Simul. Model. Pract. Theory*, vol. 123, Feb. 2023, Art. no. 102709.

[46] G. Xue, S. Deng, D. Liu, and Z. Yan, "Reaching consensus in decentralized coordination of distributed microservices," *Comput. Netw.*, vol. 187, Mar. 2021, Art. no. 107786.

[47] P. Fan, J. Liu, W. Yin, H. Wang, X. Chen, and H. Sun, "2PC: A distributed transaction concurrency control protocol of multi-microservice based on cloud computing platform," *J. Cloud Comput.*, vol. 9, no. 1, pp. 1–22, Dec. 2020.

[48] M. Gördesli, A. Nasab, and A. Varol, "Handling rollbacks with separated response control service for microservice architecture," in *Proc. 3rd Int. Informat. Softw. Eng. Conf. (IISEC)*, Dec. 2022, pp. 1–4.

[49] S. R. Vangala, B. Kasimani, and R. K. Mallidi, "Microservices event driven and streaming architectural approach for payments and trade settlement services," in *Proc. 2nd Int. Conf. Intell. Technol. (CONIT)*, Jun. 2022, pp. 1–6.

[50] D. Alur, J. Crupi, and D. Malks, *Core J2EE Patterns. Best Practices and Design Strategies*. Upper Saddle River, NJ, USA: Prentice-Hall, 2003.

[51] M. Fowler, D. Rice, M. Foemmel, E. Hieatt, R. Mee, and R. Stafford, *Patterns of Enterprise Application Architecture*. Reading, MA, USA: Addison-Wesley, 2003.

[52] A. Cockburn. (2017). *Hexagonal Architecture*. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture/

[53] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns*. New York, NY, USA: Pearson, 2015.

[54] J. Stenberg. (2014). *Exploring the Hexagonal Architecture*. InfoQ. [Online]. Available: https://www.infoq.com/news/2014/10/exploring-hexagonal-architecture/

[55] M. D. Hansen, *SOA Using Java Web Services*. Upper Saddle River, NJ, USA: Prentice-Hall, 2007.

[56] T. Erl, *SOA Design Patterns*. Upper Saddle River, NJ, USA: Prentice-Hall, 2008.

[57] A. Navarro and A. da Silva, "A metamodel-based definition of a conversion mechanism between SOAP and RESTful web services," *Comput. Standards Interface*, vol. 48, pp. 49–70, Nov. 2016.

[58] I. Sommerville, *Engineering Software Products. An Introduction to Modern Software Engineering*. Hoboken, NJ, USA: Pearson, 2020.

[59] C. Richardson. (2024). *Microservice Architecture Patterns*. [Online]. Available: https://microservices.io/patterns/microservices.html

[60] Amazon. (2024). *AWS Prescriptive Guidance*. [Online]. Available: https://aws.amazon.com/es/prescriptive-guidance/

[61] B. Posey. (2024). *Top Public Cloud Providers of 2024: A Brief Comparison*. TechTarget. [Online]. Available: https://www.techtarget.com/searchcloudcomputing/tip/Top-public-cloud-providers-A-brief-comparison

[62] *AWS Prescriptive Guidance. Building Hexagonal Architectures on AWS*. Accessed: Apr. 1, 2025. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/hexagonal-architectures/overview.html

[63] Google Cloud Architecture Centre. *Refactoring a Monolith Into Microservices*. Accessed: Apr. 1, 2025. [Online]. Available: https://cloud.google.com/architecture/microservices-architecture-refactoring-monoliths

[64] Microsoft Azure Architecture Centre. (2022). *Using Domain Analysis to Model Microservices*. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis

[65] X. Zhou, S. Li, L. Cao, H. Zhang, Z. Jia, C. Zhong, Z. Shan, and M. A. Babar, "Revisiting the practices and pains of microservice architecture in reality: An industrial inquiry," *J. Syst. Softw.*, vol. 195, Jan. 2023, Art. no. 111521.
[66] A. Navarro, W. E. Parra, E. Romero, S. Martín, and R. d. Miguel, "Securing J2EE SOA enterprise applications with a pattern-based approach," *Open J. Web Technol.*, vol. 7, no. 1, pp. 19–45, Jan. 2020. [Online]. Available: https://www.ronpub.com/OJWT_2020v7i1n02_Navarro.pdf
[67] Oracle. (2016). *Fusion Middleware Developing JAX-WS Web Services for Oracle WebLogic Server Using Web Services Atomic Transactions*. [Online]. Available: https://docs.oracle.com/middleware/12212/wls/WSGET/jax-ws-atomictransaction.htm#WSGET382
[68] S. Kostenko. (2019). *JMS Duplicate Message Detection With Wildfly*. [Online]. Available: https://kostenko.org/blog/2019/07/wildfly-jms-duplicate-detection.html
[69] ACM/IEEE (2013). *Curriculum Guidelines for Undergraduate Programs in Computer Science*. [Online]. Available: https://www.acm.org/binaries/content/assets/education/cs2013_web_final.pdf
[70] ACM/IEEE (2014). *Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering*. [Online]. Available: https://www.acm.org/binaries/content/assets/education/se2014.pdf
[71] ACM/IEEE (2010). *Curriculum Guidelines for Undergraduate Degree Programs in Information Systems*. [Online]. Available: https://www.acm.org/binaries/content/assets/education/curricula-recommendations/is-2010-acm-final.pdf
[72] ACM/IEEE (2020). *Curriculum Guidelines for Undergraduate Degree Programs in Information Systems*. [Online]. Available: https://www.acm.org/binaries/content/assets/education/curricula-recommendations/is2020.pdf
[73] A. Fox and E. A. Brewer, "Harvest, yield, and scalable tolerant systems," in *Proc. 7th Workshop Hot Topics Operating Syst.*, 1999, pp. 174–178, doi: 10.1109/HOTOS.1999.798396.
[74] K. Bhole and R. Nareddy. (2020). *Opening Banking Through Architecture Re-Engineering. A Microservices-Based Roadmap*. Deloitte. [Online]. Available: https://www2.deloitte.com/content/dam/Deloitte/us/Documents/financial-services/us-enabling-platform-banking-pov.pdf
[75] 4GVR. (2021). *Application Server Market Size, Share & Trends Analysis Report by Type (Java, Microsoft Windows), by Deployment (Hosted, On-Premise), by End Use (BFSI, Manufacturing, IT & Telecom), and Segment Forecasts, 2021–2028. Report Overview*. [Online]. Available: https://www.grandviewresearch.com/industry-analysis/application-server-market#
[76] dtm. (2021). *The Seven Most Classic Patterns for Distributed Transactions*. Deloitte. [Online]. Available: https://medium.com/@dongfuye/the-seven-most-classic-solutions-for-distributed-transaction-management-3f915f331e15
[77] A. Pandey. (2023). *Building a Hybrid Architecture: Harnessing the Power of Microservices and Monoliths*. Medium. [Online]. Available: https://medium.com/@pandeyarpit88/building-a-hybrid-architecture-harnessing-the-power-of-microservices-and-monoliths-80f79b310d75
[78] C. Steel, R. Nagappan, and R. Lai, *Core Security Patterns: Best Practices and Strategies for J2EE, Web Services, and Identity Management*. Upper Saddle River, NJ, USA: Prentice-Hall, 2005.



ANTONIO NAVARRO received the Ph.D. degree in mathematics-computer science from the Universidad Complutense de Madrid, Spain, in 2002. He is currently an Associate Professor with the Software Engineering and Artificial Intelligence Department, Universidad Complutense de Madrid. His research interests include software engineering, software architectures, software design patterns, software modelling, and model-driven architecture. He is the author and co-author of several articles related to these research topics.